

12 Appendices

12.1 Appendix A1: MCSE Yield Partition

Group 1: Model says

$$\mathbf{y}_t^1 = \mathbf{g}_0^1 + \mathbf{G}_x^{1,m} \mathbf{x}_t^m + \mathbf{G}_x^{1,l} \mathbf{x}_t^l,$$

and its lagged expression is

$$\mathbf{y}_{t-1}^1 = \mathbf{g}_0^1 + \mathbf{G}_x^{1,m} \mathbf{x}_{t-1}^m + \mathbf{G}_x^{1,l} \mathbf{x}_{t-1}^l,$$

which we can invert to get

$$\mathbf{x}_{t-1}^l = (\mathbf{G}_x^{1,l})^{-1} (\mathbf{y}_{t-1}^1 - \mathbf{g}_0^1 - \mathbf{G}_x^{1,m} \mathbf{x}_{t-1}^m).$$

Under the independence assumption the latent block under $\mathbb{P}$ has the law of motion

$$\mathbf{x}_t^l = \boldsymbol{\mu}_l^{\mathbb{P}} + \boldsymbol{\Phi}_{ll}^{\mathbb{P}} \mathbf{x}_{t-1}^l + \mathbf{u}_t^l.$$

Now substitute in for the inverted $\mathbf{x}_{t-1}^l$ expression

$$\mathbf{x}_t^l = \boldsymbol{\mu}_l^{\mathbb{P}} + \boldsymbol{\Phi}_{ll}^{\mathbb{P}} (\mathbf{G}_x^{1,l})^{-1} (\mathbf{y}_{t-1}^1 - \mathbf{g}_0^1 - \mathbf{G}_x^{1,m} \mathbf{x}_{t-1}^m) + \mathbf{u}_t^l.$$

Now plug in this expression for $\mathbf{x}_t^l$ into the model equation for $\mathbf{y}_t^1$

$$\mathbf{y}_t^1 = \mathbf{g}_0^1 + \mathbf{G}_x^{1,m} \mathbf{x}_t^m + \mathbf{G}_x^{1,l} \left[\boldsymbol{\mu}_l^{\mathbb{P}} + \boldsymbol{\Phi}_{ll}^{\mathbb{P}} (\mathbf{G}_x^{1,l})^{-1} (\mathbf{y}_{t-1}^1 - \mathbf{g}_0^1 - \mathbf{G}_x^{1,m} \mathbf{x}_{t-1}^m) + \mathbf{u}_t^l \right]$$

Now expand and collect terms

$$\begin{aligned}
\mathbf{y}_t^1 &= \mathbf{g}_0^1 + \mathbf{G}_x^{1,m} \mathbf{x}_t^m + \mathbf{G}_x^{1,l} \boldsymbol{\mu}_l^{\mathbb{P}} + \mathbf{G}_x^{1,l} \boldsymbol{\Phi}_{ll}^{\mathbb{P}} (\mathbf{G}_x^{1,l})^{-1} (\mathbf{y}_{t-1}^1 - \mathbf{g}_0^1 - \mathbf{G}_x^{1,m} \mathbf{x}_{t-1}^m) + \mathbf{G}_x^{1,l} \mathbf{u}_t^l \\
&= \mathbf{g}_0^1 + \mathbf{G}_x^{1,m} \mathbf{x}_t^m + \mathbf{G}_x^{1,l} \boldsymbol{\mu}_l^{\mathbb{P}} + \mathbf{G}_x^{1,l} \boldsymbol{\Phi}_{ll}^{\mathbb{P}} (\mathbf{G}_x^{1,l})^{-1} \mathbf{y}_{t-1}^1 - \mathbf{G}_x^{1,l} \boldsymbol{\Phi}_{ll}^{\mathbb{P}} (\mathbf{G}_x^{1,l})^{-1} \mathbf{g}_0^1 - \mathbf{G}_x^{1,l} \boldsymbol{\Phi}_{ll}^{\mathbb{P}} (\mathbf{G}_x^{1,l})^{-1} \mathbf{G}_x^{1,m} \mathbf{x}_{t-1}^m \\
&\quad + \mathbf{G}_x^{1,l} \mathbf{u}_t^l \\
&= \underbrace{\left(\mathbf{g}_0^1 + \mathbf{G}_x^{1,l} \boldsymbol{\mu}_l^{\mathbb{P}} - \mathbf{G}_x^{1,l} \boldsymbol{\Phi}_{ll}^{\mathbb{P}} (\mathbf{G}_x^{1,l})^{-1} \mathbf{g}_0^1 \right)}_{\mathbf{A}^{1,\#}} + \underbrace{\mathbf{G}_x^{1,l} \boldsymbol{\Phi}_{ll}^{\mathbb{P}} (\mathbf{G}_x^{1,l})^{-1} \mathbf{y}_{t-1}^1}_{\boldsymbol{\Phi}_{11}^{\mathbb{P},\#}} + \underbrace{\left(\mathbf{G}_x^{1,m} \mathbf{x}_t^m - \mathbf{G}_x^{1,l} \boldsymbol{\Phi}_{ll}^{\mathbb{P}} (\mathbf{G}_x^{1,l})^{-1} \mathbf{G}_x^{1,m} \mathbf{x}_{t-1}^m \right)}_{\boldsymbol{\Psi}_{1m}^{\mathbb{P},\#}} \\
&\quad + \mathbf{G}_x^{1,l} \mathbf{u}_t^l \\
&= \mathbf{A}^{1,\#} + \boldsymbol{\Phi}_{11}^{\mathbb{P},\#} \mathbf{y}_{t-1}^1 + \boldsymbol{\Psi}_{1m}^{\mathbb{P},\#} \mathbf{x}_t^m - \mathbf{G}_x^{1,l} \boldsymbol{\Phi}_{ll}^{\mathbb{P}} (\mathbf{G}_x^{1,l})^{-1} \mathbf{G}_x^{1,m} \mathbf{x}_{t-1}^m + \mathbf{G}_x^{1,l} \mathbf{u}_t^l.
\end{aligned}$$

The term $\mathbf{G}_x^{1,l} \boldsymbol{\Phi}_{ll}^{\mathbb{P}} (\mathbf{G}_x^{1,l})^{-1} \mathbf{G}_x^{1,m} \mathbf{x}_{t-1}^m$ is a lagged-macro loading. It coincides with ϕ_{m1}^* in the exactly-priced Eq. (36) in [Hamilton and Wu \(2012\)](#). The block-independence assumption $\boldsymbol{\Phi}_{lm}^{\mathbb{P}} = \boldsymbol{\Phi}_{ml}^{\mathbb{P}} = \mathbf{0}$ removes the direct feedback channel given by $\mathbf{G}_x^{1,l} \boldsymbol{\Phi}_{lm}^{\mathbb{P}}$, but the inversion channel remains, and the term is generally non-zero whenever the exactly-priced yields load on the macro factor $(\mathbf{G}_x^{1,m}) \neq \mathbf{0}$. Because the exactly-priced yields load only very weakly on the macro factor in our estimates ($\beta_m \approx \pm 0.0007$, so $\mathbf{G}_x^{1,m}$ is small), the coefficient on this lagged-macro term, $\boldsymbol{\Phi}_{11}^{\mathbb{P},\#} \mathbf{G}_x^{1,m}$, is negligible. We therefore drop it and work with the approximate Group 1 reduced form. We are then left with

$$\mathbf{y}_t^1 = \mathbf{A}^{1,\#} + \boldsymbol{\Phi}_{11}^{\mathbb{P},\#} \mathbf{y}_{t-1}^1 + \boldsymbol{\Psi}_{1m}^{\mathbb{P},\#} \mathbf{x}_t^m + \mathbf{G}_x^{1,l} \mathbf{u}_t^l.$$

Group 2: Model says

$$\mathbf{y}_t^2 = \mathbf{g}_0^2 + \mathbf{G}_x^{2,m} \mathbf{x}_t^m + \mathbf{G}_x^{2,l} \mathbf{x}_t^l.$$

The yield equation for group 1 was given by

$$\mathbf{y}_t^1 = \mathbf{g}_0^1 + \mathbf{G}_x^{1,m} \mathbf{x}_t^m + \mathbf{G}_x^{1,l} \mathbf{x}_t^l.$$

Since $\mathbf{G}_x^{1,l}$ is an 2×2 invertible matrix by construction (two maturities are chosen precisely to span the latent factor space), we can solve for the latent factors

$$\mathbf{x}_t^l = (\mathbf{G}_x^{1,l})^{-1} (\mathbf{y}_t^1 - \mathbf{g}_0^1 - \mathbf{G}_x^{1,m} \mathbf{x}_t^m).$$

Inserting this expression into $\mathbf{y}_t^2$ yields

$$\mathbf{y}_t^2 = \mathbf{g}_0^2 + \mathbf{G}_x^{2,m} \mathbf{x}_t^m + \mathbf{G}_x^{2,l} (\mathbf{G}_x^{1,l})^{-1} (\mathbf{y}_t^1 - \mathbf{g}_0^1 - \mathbf{G}_x^{1,m} \mathbf{x}_t^m).$$

Now expand and collect terms

$$\begin{aligned} \mathbf{y}_t^2 &= \mathbf{g}_0^2 + \mathbf{G}_x^{2,m} \mathbf{x}_t^m + \mathbf{G}_x^{2,l} (\mathbf{G}_x^{1,l})^{-1} \mathbf{y}_t^1 - \mathbf{G}_x^{2,l} (\mathbf{G}_x^{1,l})^{-1} \mathbf{g}_0^1 - \mathbf{G}_x^{2,l} (\mathbf{G}_x^{1,l})^{-1} \mathbf{G}_x^{1,m} \mathbf{x}_t^m \\ &= \underbrace{(\mathbf{g}_0^2 - \mathbf{G}_x^{2,l} (\mathbf{G}_x^{1,l})^{-1} \mathbf{g}_0^1)}_{\mathbf{A}^{2,\#}} + \underbrace{(\mathbf{G}_x^{2,m} - \mathbf{G}_x^{2,l} (\mathbf{G}_x^{1,l})^{-1} \mathbf{G}_x^{1,m})}_{\Phi_{2m}^{\mathbb{P},\#}} \mathbf{x}_t^m + \underbrace{\mathbf{G}_x^{2,l} (\mathbf{G}_x^{1,l})^{-1}}_{\Phi_{21}^{\mathbb{P},\#}} \mathbf{y}_t^1 \\ &= \mathbf{A}^{2,\#} + \Phi_{2m}^{\mathbb{P},\#} \mathbf{x}_t^m + \Phi_{21}^{\mathbb{P},\#} \mathbf{y}_t^1. \end{aligned}$$

12.2 Appendix A2: MCSE Conditions

Condition 2:

Take

$$\mathbf{y}_t^2 = \mathbf{A}^{2,\#} + \Phi_{2m}^{\mathbb{P},\#} \mathbf{x}_t^m + \Phi_{21}^{\mathbb{P},\#} \mathbf{y}_t^1 + \mathbf{u}_t^{2,\#}.$$

Now insert for the yield expression for $\mathbf{y}_t^1$

$$\mathbf{y}_t^2 = \mathbf{A}^{2,\#} + \Phi_{2m}^{\mathbb{P},\#} \mathbf{x}_t^m + \Phi_{21}^{\mathbb{P},\#} (\mathbf{g}_0^1 + \mathbf{G}_x^{1,m} \mathbf{x}_t^m + \mathbf{G}_x^{1,l} \mathbf{x}_t^l) + \mathbf{u}_t^{2,\#}.$$

Now expand and collect terms

$$\begin{aligned} \mathbf{y}_t^2 &= \mathbf{A}^{2,\#} + \Phi_{2m}^{\mathbb{P},\#} \mathbf{x}_t^m + \Phi_{21}^{\mathbb{P},\#} \mathbf{g}_0^1 + \Phi_{21}^{\mathbb{P},\#} \mathbf{G}_x^{1,m} \mathbf{x}_t^m + \Phi_{21}^{\mathbb{P},\#} \mathbf{G}_x^{1,l} \mathbf{x}_t^l + \mathbf{u}_t^{2,\#} \\ &= (\mathbf{A}^{2,\#} + \Phi_{21}^{\mathbb{P},\#} \mathbf{g}_0^1) + (\Phi_{2m}^{\mathbb{P},\#} + \Phi_{21}^{\mathbb{P},\#} \mathbf{G}_x^{1,m}) \mathbf{x}_t^m + \Phi_{21}^{\mathbb{P},\#} \mathbf{G}_x^{1,l} \mathbf{x}_t^l + \mathbf{u}_t^{2,\#} \\ &= (\mathbf{A}^{2,\#} + \Phi_{21}^{\mathbb{P},\#} \mathbf{g}_0^1) + (\Phi_{2m}^{\mathbb{P},\#} + \Phi_{21}^{\mathbb{P},\#} \widehat{\Psi}_{1m}^{\mathbb{P},\#}) \mathbf{x}_t^m + \Phi_{21}^{\mathbb{P},\#} \mathbf{G}_x^{1,l} \mathbf{x}_t^l + \mathbf{u}_t^{2,\#}. \end{aligned}$$

The second MCSE condition is then given by

$$\mathbf{G}_{x,OLS}^{2,m} = \Phi_{2m}^{\mathbb{P},\#} + \Phi_{21}^{\mathbb{P},\#} \widehat{\Psi}_{1m}^{\mathbb{P},\#}.$$

Condition 3: With

$$\mathbf{y}_t^1 = \mathbf{A}^{1,\#} + \Phi_{11}^{\mathbb{P},\#} \mathbf{y}_{t-1}^1 + \Psi_{1m}^{\mathbb{P},\#} \mathbf{x}_t^m + \mathbf{u}_t^{1,\#},$$

After controlling for both $\mathbf{y}_{t-1}^1$ and $\mathbf{x}_t^m$, the residuals $\mathbf{u}_t^{1,\#}$ only absorb the latent factor

innovations

$$\widehat{\Omega}^{1,\#} = \frac{1}{T_1} \sum_t \mathbf{u}_t^{1,\#} \left(\mathbf{u}_t^{1,\#} \right)^\top = \mathbf{G}_x^{1,l} \Sigma_{ll} \left(\mathbf{G}_x^{1,l} \right)^\top = \mathbf{G}_x^{1,l} \left(\mathbf{G}_x^{1,l} \right)^\top,$$

where it is used that $\Sigma_{ll} = \mathbf{I}$.

The third MCSE condition is then

$$\widehat{\Omega}^{1,\#} = \mathbf{G}_x^{1,l} \left(\mathbf{G}_x^{1,l} \right)^\top.$$

Condition 4: Looking at the expression used in condition 2 and comparing coefficients, the fourth condition must be given by

$$\mathbf{G}_x^{2,l} = \Phi_{21}^{\mathbb{P},\#} \mathbf{G}_x^{1,l},$$

but since $\mathbf{G}_x^{1,l}$ is unobserved we first multiply both sides by $\left(\mathbf{G}_x^{1,l} \right)^\top$

$$\mathbf{G}_x^{2,l} \left(\mathbf{G}_x^{1,l} \right)^\top = \Phi_{21}^{\mathbb{P},\#} \mathbf{G}_x^{1,l} \left(\mathbf{G}_x^{1,l} \right)^\top.$$

The fourth condition is then given by

$$\mathbf{G}_{x,OLS}^{2,l} \left(\mathbf{G}_x^{1,l} \right)^\top = \Phi_{21}^{\mathbb{P},\#} \widehat{\Omega}^{1,\#}.$$

12.3 Appendix A3: $\mathbb{P}$ to $\mathbb{Q}$

For a Gaussian innovation we can move from the $\mathbb{P}$ to the $\mathbb{Q}$ -measure by using the Radon-Nikodym derivative, given by

$$\frac{d\mathbb{Q}}{d\mathbb{P}} = \exp \left\{ -\frac{1}{2} \Lambda_t^\top \Lambda_t - \boldsymbol{\varepsilon}_{t+1}^\top \Lambda_t \right\}.$$

The density for $\boldsymbol{\varepsilon}_{t+1}$ under $\mathbb{P}$ is simply the multivariate Gaussian density

$$\begin{aligned} \mathbb{P}(\boldsymbol{\varepsilon}_{t+1}) &= (2\pi)^{-\frac{F}{2}} \det(\mathbf{I})^{-\frac{1}{2}} \exp \left(-\frac{1}{2} (\boldsymbol{\varepsilon}_{t+1} - \mathbf{0})^\top \mathbf{I}^{-1} (\boldsymbol{\varepsilon}_{t+1} - \mathbf{0}) \right) \\ &= (2\pi)^{-\frac{F}{2}} \exp \left(-\frac{1}{2} \boldsymbol{\varepsilon}_{t+1}^\top \boldsymbol{\varepsilon}_{t+1} \right). \end{aligned}$$

The density for $\boldsymbol{\varepsilon}_{t+1}$ under $\mathbb{Q}$ can then be found by $\mathbb{Q}(\boldsymbol{\varepsilon}_{t+1}) = \mathbb{P}(\boldsymbol{\varepsilon}_{t+1}) \times d\mathbb{Q}/d\mathbb{P}$, giving us

$$\mathbb{Q}(\boldsymbol{\varepsilon}_{t+1}) = (2\pi)^{-\frac{F}{2}} \exp\left(-\frac{1}{2}\boldsymbol{\varepsilon}_{t+1}^\top \boldsymbol{\varepsilon}_{t+1}\right) \exp\left\{-\frac{1}{2}\boldsymbol{\Lambda}_t^\top \boldsymbol{\Lambda}_t - \boldsymbol{\varepsilon}_{t+1}^\top \boldsymbol{\Lambda}_t\right\},$$

which can be rewritten as

$$\mathbb{Q}(\boldsymbol{\varepsilon}_{t+1}) = (2\pi)^{-\frac{F}{2}} \exp\left(-\frac{1}{2}[\boldsymbol{\varepsilon}_{t+1} + \boldsymbol{\Lambda}_t]^\top [\boldsymbol{\varepsilon}_{t+1} + \boldsymbol{\Lambda}_t]\right).$$

By looking at the differences between the densities under $\mathbb{P}$ and $\mathbb{Q}$ it shows that our Gaussian innovation under $\mathbb{Q}$ is given by

$$\boldsymbol{\varepsilon}_{t+1}^{\mathbb{Q}} = \boldsymbol{\varepsilon}_{t+1}^{\mathbb{P}} + \boldsymbol{\Lambda}_t.$$

Furthermore, using the above result we can derive

$$\begin{aligned} \mathbf{x}_{t+1} &= \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t + \boldsymbol{\Sigma} (\boldsymbol{\varepsilon}_{t+1}^{\mathbb{Q}} - \boldsymbol{\Lambda}_t) \\ &= \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t + \boldsymbol{\Sigma} \boldsymbol{\varepsilon}_{t+1}^{\mathbb{Q}} - \boldsymbol{\Sigma} \boldsymbol{\Lambda}_t \\ &= \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t + \boldsymbol{\Sigma} \boldsymbol{\varepsilon}_{t+1}^{\mathbb{Q}} - \boldsymbol{\Sigma} \boldsymbol{\Sigma}^{-1} (\boldsymbol{\lambda}_0 + \boldsymbol{\lambda}_x \mathbf{x}_t) \\ &= \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t + \boldsymbol{\Sigma} \boldsymbol{\varepsilon}_{t+1}^{\mathbb{Q}} - \boldsymbol{\lambda}_0 - \boldsymbol{\lambda}_x \mathbf{x}_t \\ &= \underbrace{(\boldsymbol{\mu}^{\mathbb{P}} - \boldsymbol{\lambda}_0)}_{\boldsymbol{\mu}^{\mathbb{Q}}} + \underbrace{(\boldsymbol{\Phi}^{\mathbb{P}} - \boldsymbol{\lambda}_x)}_{\boldsymbol{\Phi}^{\mathbb{Q}}} \mathbf{x}_t + \boldsymbol{\Sigma} \boldsymbol{\varepsilon}_{t+1}^{\mathbb{Q}} \\ \mathbf{x}_{t+1} &= \boldsymbol{\mu}^{\mathbb{Q}} + \boldsymbol{\Phi}^{\mathbb{Q}} \mathbf{x}_t + \boldsymbol{\Sigma} \boldsymbol{\varepsilon}_{t+1}^{\mathbb{Q}}. \end{aligned}$$

12.4 Appendix A4: FEVD

Yields are given by

$$y_t^{(k)} = g_0^{(k)} + \mathbf{G}_x^{(k)} \mathbf{x}_t,$$

and the state dynamics are

$$\mathbf{x}_{t+1} = \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t + \boldsymbol{\Sigma} \boldsymbol{\varepsilon}_{t+1}^{\mathbb{P}}.$$

The IRF for yields given horizon h , maturity k , and structural shock j is

$$\text{IRF}_y^{(k,j)}(h) = \mathbf{G}_x^{(k)} (\boldsymbol{\Phi}^{\mathbb{P}})^{h-1} \boldsymbol{\Sigma} \mathbf{e}_j.$$

Use the fact that a stationary AR(1) process can be represented by a MA(∞) process, so that the state vector has the following MA representation

$$\mathbf{x}_t - \underbrace{\bar{\mathbf{x}}}_{(\mathbf{I} - \Phi^{\mathbb{P}})^{-1} \mu^{\mathbb{P}}} = \sum_{i=0}^{\infty} (\Phi^{\mathbb{P}})^i \Sigma \varepsilon_{t-i}^{\mathbb{P}}.$$

Since yields are affine in $\mathbf{x}_t$ we can write

$$y_t^{(k)} - \bar{y}^{(k)} = \mathbf{G}_x^{(k)} (\mathbf{x}_t - \bar{\mathbf{x}}) = \sum_{i=0}^{\infty} \mathbf{G}_x^{(k)} (\Phi^{\mathbb{P}})^i \Sigma \varepsilon_{t-i}^{\mathbb{P}}.$$

Now define $\Psi_{i,k,j} \equiv \mathbf{G}_x^{(k)} (\Phi^{\mathbb{P}})^i \Sigma \mathbf{e}_j$ as the (k, j) element of the MA coefficient matrix $\Psi_i = \mathbf{G}_x (\Phi^{\mathbb{P}})^i \Sigma$ at i . The h -step ahead forecast error for yield $y_t^{(k)}$ is

$$\begin{aligned} y_{t+h}^{(k)} - y_{t+h|t}^{(k)} &= \sum_{i=0}^{h-1} \mathbf{G}_x^{(k)} (\Phi^{\mathbb{P}})^i \Sigma \varepsilon_{t+h-i}^{\mathbb{P}} \\ &= \sum_{i=0}^{h-1} \sum_{j=1}^4 \Psi_{i,k,j} \varepsilon_{t+h-i,j}^{\mathbb{P}}. \end{aligned}$$

Since $\varepsilon^{\mathbb{P}} \sim \text{iid}(\mathbf{0}, \mathbf{I})$, all shocks are serially uncorrelated and mutually orthogonal. The mean squared error of the forecast error h steps ahead for the yield at maturity k is then

$$\text{MSE}^{(k)}(h) = \text{Var} \left(\sum_{i=0}^{h-1} \sum_{j=1}^4 \Psi_{i,k,j} \varepsilon_{t+h-i,j}^{\mathbb{P}} \right) = \sum_{j=1}^4 \sum_{i=0}^{h-1} \Psi_{i,k,j}^2.$$

The portion of $\text{MSE}^{(k)}(h)$ that can attributed solely to shock j is

$$\text{MSE}_j^{(k)}(h) = \sum_{i=0}^{h-1} \Psi_{i,k,j}^2.$$

The share of total forecast variance attributable to shock j is

$$\Omega_{k,j}(h) = \frac{\text{MSE}_j^{(k)}(h)}{\text{MSE}^{(k)}(h)} = \frac{\sum_{i=0}^{h-1} \Psi_{i,k,j}^2}{\sum_{j=1}^4 \sum_{i=0}^{h-1} \Psi_{i,k,j}^2}.$$

12.5 Appendix A5: $\mathbb{P}$ Recursion

The law of motion under the $\mathbb{P}$ -measure is given by

$$\mathbf{x}_{t+1} = \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t + \boldsymbol{\Sigma} \boldsymbol{\varepsilon}_{t+1}^{\mathbb{P}}.$$

The conditional expectation under $\mathbb{P}$ at time t is

$$\mathbb{E}_t^{\mathbb{P}} [\mathbf{x}_{t+1}] = \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t.$$

Now iterate forward. The two period ahead ($j = 2$) is

$$\begin{aligned} \mathbb{E}_t^{\mathbb{P}} [\mathbf{x}_{t+2}] &= \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbb{E}_t^{\mathbb{P}} [\mathbf{x}_{t+1}] \\ &= \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbb{E}_t^{\mathbb{P}} [\boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t] \\ &= \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} [\boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t] \\ &= \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \boldsymbol{\mu}^{\mathbb{P}} + (\boldsymbol{\Phi}^{\mathbb{P}})^2 \mathbf{x}_t \\ &= (\mathbf{I} + \boldsymbol{\Phi}^{\mathbb{P}}) \boldsymbol{\mu}^{\mathbb{P}} + (\boldsymbol{\Phi}^{\mathbb{P}})^2 \mathbf{x}_t. \end{aligned}$$

For $j = 3$

$$\begin{aligned} \mathbb{E}_t^{\mathbb{P}} [\mathbf{x}_{t+3}] &= \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbb{E}_t^{\mathbb{P}} [\mathbf{x}_{t+2}] \\ &= \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbb{E}_t^{\mathbb{P}} [(\mathbf{I} + \boldsymbol{\Phi}^{\mathbb{P}}) \boldsymbol{\mu}^{\mathbb{P}} + (\boldsymbol{\Phi}^{\mathbb{P}})^2 \mathbf{x}_t] \\ &= \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} [(\mathbf{I} + \boldsymbol{\Phi}^{\mathbb{P}}) \boldsymbol{\mu}^{\mathbb{P}} + (\boldsymbol{\Phi}^{\mathbb{P}})^2 \mathbf{x}_t] \\ &= \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \boldsymbol{\mu}^{\mathbb{P}} + (\boldsymbol{\Phi}^{\mathbb{P}})^2 \boldsymbol{\mu}^{\mathbb{P}} + (\boldsymbol{\Phi}^{\mathbb{P}})^3 \mathbf{x}_t \\ &= (\mathbf{I} + \boldsymbol{\Phi}^{\mathbb{P}} + (\boldsymbol{\Phi}^{\mathbb{P}})^2) \boldsymbol{\mu}^{\mathbb{P}} + (\boldsymbol{\Phi}^{\mathbb{P}})^3 \mathbf{x}_t. \end{aligned}$$

Now the pattern is clear and after j steps we have

$$\begin{aligned} \mathbb{E}_t^{\mathbb{P}} [\mathbf{x}_{t+i}] &= (\mathbf{I} + \boldsymbol{\Phi}^{\mathbb{P}} + (\boldsymbol{\Phi}^{\mathbb{P}})^2 + \dots + (\boldsymbol{\Phi}^{\mathbb{P}})^{i-1}) \boldsymbol{\mu}^{\mathbb{P}} + (\boldsymbol{\Phi}^{\mathbb{P}})^i \mathbf{x}_t \\ \mathbb{E}_t^{\mathbb{P}} [\mathbf{x}_{t+i}] &= \sum_{j=0}^{i-1} (\boldsymbol{\Phi}^{\mathbb{P}})^j \boldsymbol{\mu}^{\mathbb{P}} + (\boldsymbol{\Phi}^{\mathbb{P}})^i \mathbf{x}_t. \end{aligned}$$

12.6 Appendix A6: Alternative Term Premium

The term premium for a given maturity k is given as the wedge between the yield and the average of the expected future short-rates

$$TP_t^{(k)} = y_t^{(k)} - \frac{1}{k} \sum_{i=0}^{k-1} \mathbb{E}_t^{\mathbb{P}} [r_{t+i}].$$

One-period expected excess return. If one buys a k -period bond at time t and then sells it as a $(k-1)$ -period bond at $t+1$, the one-period log holding return in excess of the short-rate is

$$rx_{t+1}^{(k)} \equiv \log P_{t+1}^{(k-1)} - \log P_t^{(k)} - r_t = A_{k-1} + \mathbf{B}_{k-1}^\top \mathbf{x}_{t+1} - A_k - \mathbf{B}_k^\top \mathbf{x}_t - r_t.$$

Take the conditional expectation under $\mathbb{P}$ and insert for $\mathbb{E}_t^{\mathbb{P}} [\mathbf{x}_{t+1}] = \boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t$ and $r_t = \alpha + \boldsymbol{\beta}^\top \mathbf{x}_t$.

$$\begin{aligned} \mathbb{E}_t^{\mathbb{P}} [rx_{t+1}^{(k)}] &= \mathbb{E}_t^{\mathbb{P}} [A_{k-1} + \mathbf{B}_{k-1}^\top \mathbf{x}_{t+1} - A_k - \mathbf{B}_k^\top \mathbf{x}_t - r_t] \\ &= (A_{k-1} - A_k) + \mathbf{B}_{k-1}^\top \mathbb{E}_t^{\mathbb{P}} [\mathbf{x}_{t+1}] - \mathbf{B}_k^\top \mathbf{x}_t - r_t \\ &= (A_{k-1} - A_k) + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t) - \mathbf{B}_k^\top \mathbf{x}_t - \alpha - \boldsymbol{\beta}^\top \mathbf{x}_t. \end{aligned}$$

Now insert (15) and (16)

$$\begin{aligned} \mathbb{E}_t^{\mathbb{P}} [rx_{t+1}^{(k)}] &= \left(\alpha - \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^{\mathbb{P}} - \boldsymbol{\lambda}_0) - \frac{1}{2} \mathbf{B}_{k-1}^\top \boldsymbol{\Sigma} \boldsymbol{\Sigma}^\top \mathbf{B}_{k-1} \right) + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t) \\ &\quad - (\mathbf{B}_{k-1}^\top (\boldsymbol{\Phi}^{\mathbb{P}} - \boldsymbol{\lambda}_x) - \boldsymbol{\beta}^\top) \mathbf{x}_t - \alpha - \boldsymbol{\beta}^\top \mathbf{x}_t \\ &= -\mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^{\mathbb{P}} - \boldsymbol{\lambda}_0) - \frac{1}{2} \mathbf{B}_{k-1}^\top \boldsymbol{\Sigma} \boldsymbol{\Sigma}^\top \mathbf{B}_{k-1} + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^{\mathbb{P}} + \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t) - \mathbf{B}_{k-1}^\top (\boldsymbol{\Phi}^{\mathbb{P}} - \boldsymbol{\lambda}_x) \mathbf{x}_t \\ &= -\mathbf{B}_{k-1}^\top \boldsymbol{\mu}^{\mathbb{P}} + \mathbf{B}_{k-1}^\top \boldsymbol{\lambda}_0 - \frac{1}{2} \mathbf{B}_{k-1}^\top \boldsymbol{\Sigma} \boldsymbol{\Sigma}^\top \mathbf{B}_{k-1} + \mathbf{B}_{k-1}^\top \boldsymbol{\mu}^{\mathbb{P}} + \mathbf{B}_{k-1}^\top \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t - \mathbf{B}_{k-1}^\top \boldsymbol{\Phi}^{\mathbb{P}} \mathbf{x}_t + \mathbf{B}_{k-1}^\top \boldsymbol{\lambda}_x \mathbf{x}_t \\ &= \mathbf{B}_{k-1}^\top \boldsymbol{\lambda}_0 - \frac{1}{2} \mathbf{B}_{k-1}^\top \boldsymbol{\Sigma} \boldsymbol{\Sigma}^\top \mathbf{B}_{k-1} + \mathbf{B}_{k-1}^\top \boldsymbol{\lambda}_x \mathbf{x}_t \\ &= \mathbf{B}_{k-1}^\top (\boldsymbol{\lambda}_0 + \boldsymbol{\lambda}_x \mathbf{x}_t) - \frac{1}{2} \mathbf{B}_{k-1}^\top \boldsymbol{\Sigma} \boldsymbol{\Sigma}^\top \mathbf{B}_{k-1} \\ &= \mathbf{B}_{k-1}^\top \boldsymbol{\Sigma} \boldsymbol{\Lambda}_t - \frac{1}{2} \mathbf{B}_{k-1}^\top \boldsymbol{\Sigma} \boldsymbol{\Sigma}^\top \mathbf{B}_{k-1}. \end{aligned}$$

The first term $\mathbf{B}_{k-1}^\top \boldsymbol{\Sigma} \boldsymbol{\Lambda}_t$ is the risk premium and the second is the Jensen convexity correction which adjusts for the bias when using expected values with non-linear functions.

Term Premium. Holding the k -period bond to maturity and rolling it down one period at

a time the log holding returns is $-\log P_t^{(k)} = ky_t^{(k)}$ since $P_{t+k}^{(0)} = 1$

$$ky_t^{(k)} = \sum_{i=1}^k \left(\log P_{t+i}^{(k-i)} - \log P_{t+i-1}^{(k-i+1)} \right).$$

Now insert for the expected one-period excess return

$$ky_t^{(k)} = \sum_{i=1}^k \left(rx_{t+i}^{(k-i+1)} + r_{t+i-1} \right).$$

Now that the conditional expectation under the physical measure and rearrange a bit

$$\begin{aligned} \mathbb{E}_t^{\mathbb{P}} \left[ky_t^{(k)} \right] &= \sum_{i=1}^k \left(\mathbb{E}_t^{\mathbb{P}} \left[rx_{t+i}^{(k-i+1)} \right] + \mathbb{E}_t^{\mathbb{P}} \left[r_{t+i-1} \right] \right) \\ \Leftrightarrow y_t^{(k)} &= \frac{1}{k} \sum_{i=1}^k \mathbb{E}_t^{\mathbb{P}} \left[rx_{t+i}^{(k-i+1)} \right] + \frac{1}{k} \sum_{i=1}^k \mathbb{E}_t^{\mathbb{P}} \left[r_{t+i-1} \right] \\ y_t^{(k)} &= \frac{1}{k} \sum_{i=0}^{k-1} \mathbb{E}_t^{\mathbb{P}} \left[r_{t+i} \right] + \frac{1}{k} \sum_{i=1}^k \mathbb{E}_t^{\mathbb{P}} \left[rx_{t+i}^{(k-i+1)} \right]. \end{aligned}$$

Now by using the definition of the term premium we see that

$$TP_t^{(k)} = \frac{1}{k} \sum_{i=1}^k \mathbb{E}_t^{\mathbb{P}} \left[rx_{t+i}^{(k-i+1)} \right],$$

must be the term premium in alternative form.

12.7 Appendix A7: Solving the Model

Non-arbitrage condition.

For $k = 1$

$$\begin{aligned} P_t^{(1)} &= \mathbb{E} \left[M_{t+1} P_{t+1}^{(0)} \right] \\ &= \mathbb{E} [M_{t+1}] . \end{aligned}$$

For $k = 2$

$$\begin{aligned} P_t^{(2)} &= \mathbb{E} \left[M_{t+1} P_{t+1}^{(1)} \right] \\ &= \mathbb{E} [M_{t+1} M_{t+2}] . \end{aligned}$$

For $k = 3$

$$\begin{aligned} P_t^{(3)} &= \mathbb{E} \left[M_{t+1} P_{t+1}^{(2)} \right] \\ &= \mathbb{E} [M_{t+1} M_{t+2} M_{t+3}] . \end{aligned} \tag{92}$$

We see the pattern and generally for a given k the price of a bond is

$$P_t^{(k)} = \mathbb{E} \left[\prod_{i=0}^{k-1} M_{t+1+i} \times 1 \right] .$$

Solving for bond yields. Assume that the price of zero-coupon bonds is given by the affine equation

$$P_t^{(k)} = \exp \{ A_k + B_k^\top \mathbf{x}_t \} ,$$

Since

$$P_t^{(k)} = \exp (A_k + \mathbf{B}_k^\top \mathbf{x}_t) ,$$

we then have that

$$\begin{aligned} \exp (A_k + \mathbf{B}_k^\top \mathbf{x}_t) &= \mathbb{E}_t \left(M_{t+1} P_{t+1}^{(k-1)} \right) \\ &= \mathbb{E}_t \left(\exp \left\{ -r_t - \frac{1}{2} \boldsymbol{\Lambda}_t^\top \boldsymbol{\Lambda}_t - \boldsymbol{\Lambda}_t^\top \boldsymbol{\epsilon}_{t+1}^\mathbb{P} \right\} \exp \{ A_{k-1} + \mathbf{B}_{k-1}^\top \mathbf{x}_{t+1} \} \right) . \end{aligned}$$

Now inserting Eq. (6) we get

$$\begin{aligned}
\exp(A_k + \mathbf{B}_k^\top \mathbf{x}_t) &= \mathbb{E}_t \left(\exp \left\{ -r_t - \frac{1}{2} \mathbf{\Lambda}_t^\top \mathbf{\Lambda}_t - \mathbf{\Lambda}_t^\top \boldsymbol{\varepsilon}_{t+1}^\mathbb{P} \right\} \exp \{ A_{k-1} + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^\mathbb{P} + \Phi^\mathbb{P} \mathbf{x}_t + \Sigma \boldsymbol{\varepsilon}_{t+1}^\mathbb{P}) \} \right) \\
&= \mathbb{E}_t \left(\exp \left\{ A_{k-1} - r_t - \frac{1}{2} \mathbf{\Lambda}_t^\top \mathbf{\Lambda}_t \right\} \exp \{ -\mathbf{\Lambda}_t^\top \boldsymbol{\varepsilon}_{t+1}^\mathbb{P} + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^\mathbb{P} + \Phi^\mathbb{P} \mathbf{x}_t + \Sigma \boldsymbol{\varepsilon}_{t+1}^\mathbb{P}) \} \right) \\
&= \exp \left(A_{k-1} - r_t - \frac{1}{2} \mathbf{\Lambda}_t^\top \mathbf{\Lambda}_t + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^\mathbb{P} + \Phi^\mathbb{P} \mathbf{x}_t) \right) \mathbb{E}_t \left(\exp (-\mathbf{\Lambda}_t^\top \boldsymbol{\varepsilon}_{t+1}^\mathbb{P} + \mathbf{B}_{k-1}^\top \Sigma \boldsymbol{\varepsilon}_{t+1}^\mathbb{P}) \right).
\end{aligned}$$

Now note that for any multivariate random variable $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$, the moment-generation function (MGF) is given by

$$\mathbf{M}_{\mathbf{X}}(\boldsymbol{\tau}) = \exp \left\{ \boldsymbol{\tau}^\top \boldsymbol{\mu} + \frac{1}{2} \boldsymbol{\tau}^\top \Sigma \boldsymbol{\tau} \right\},$$

where $\boldsymbol{\tau}$ is a conformable vector. For $\boldsymbol{\varepsilon}_{t+1}^\mathbb{P} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and letting $\mathbf{B}_{k-1}^\top \Sigma - \mathbf{\Lambda}_t^\top$ be the conformable vector we can rewrite the expectation term as

$$\begin{aligned}
\mathbb{E}_t \left(\exp (-\mathbf{\Lambda}_t^\top \boldsymbol{\varepsilon}_{t+1}^\mathbb{P} + \mathbf{B}_{k-1}^\top \Sigma \boldsymbol{\varepsilon}_{t+1}^\mathbb{P}) \right) &= \exp \left\{ \frac{1}{2} (\mathbf{B}_{k-1}^\top \Sigma - \mathbf{\Lambda}_t^\top) (\mathbf{B}_{k-1}^\top \Sigma - \mathbf{\Lambda}_t^\top)^\top \right\} \\
&= \exp \left\{ \frac{1}{2} (\mathbf{B}_{k-1}^\top \Sigma \Sigma^\top \mathbf{B}_{k-1} - \mathbf{B}_{k-1}^\top \Sigma \mathbf{\Lambda}_t - \mathbf{\Lambda}_t^\top \Sigma^\top \mathbf{B}_{k-1} + \mathbf{\Lambda}_t^\top \mathbf{\Lambda}_t) \right\} \\
&= \exp \left\{ \frac{1}{2} (\mathbf{B}_{k-1}^\top \Sigma \Sigma^\top \mathbf{B}_{k-1} - 2\mathbf{B}_{k-1}^\top \Sigma \mathbf{\Lambda}_t + \mathbf{\Lambda}_t^\top \mathbf{\Lambda}_t) \right\}.
\end{aligned}$$

The full expression is then

$$\begin{aligned}
\exp(A_k + \mathbf{B}_k^\top \mathbf{x}_t) &= \exp \left(A_{k-1} - r_t - \frac{1}{2} \mathbf{\Lambda}_t^\top \mathbf{\Lambda}_t + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^\mathbb{P} + \Phi^\mathbb{P} \mathbf{x}_t) \right) \\
&\quad \times \exp \left(\frac{1}{2} (\mathbf{B}_{k-1}^\top \Sigma \Sigma^\top \mathbf{B}_{k-1} - 2\mathbf{B}_{k-1}^\top \Sigma \mathbf{\Lambda}_t + \mathbf{\Lambda}_t^\top \mathbf{\Lambda}_t) \right).
\end{aligned}$$

Taking logs then yields

$$A_k + \mathbf{B}_k^\top \mathbf{x}_t = A_{k-1} - r_t - \frac{1}{2} \mathbf{\Lambda}_t^\top \mathbf{\Lambda}_t + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^\mathbb{P} + \Phi^\mathbb{P} \mathbf{x}_t) + \frac{1}{2} (\mathbf{B}_{k-1}^\top \Sigma \Sigma^\top \mathbf{B}_{k-1} - 2\mathbf{B}_{k-1}^\top \Sigma \mathbf{\Lambda}_t + \mathbf{\Lambda}_t^\top \mathbf{\Lambda}_t).$$

We then simply rewrite a bit

$$\begin{aligned}
&= A_{k-1} - r_t - \frac{1}{2} \mathbf{\Lambda}_t^\top \mathbf{\Lambda}_t + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^\mathbb{P} + \boldsymbol{\Phi}^\mathbb{P} \mathbf{x}_t) + \frac{1}{2} \mathbf{B}_{k-1}^\top \Sigma \Sigma^\top \mathbf{B}_{k-1} - \mathbf{B}_{k-1}^\top \Sigma \mathbf{\Lambda}_t + \frac{1}{2} \mathbf{\Lambda}_t^\top \mathbf{\Lambda}_t \\
&= A_{k-1} - r_t + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^\mathbb{P} + \boldsymbol{\Phi}^\mathbb{P} \mathbf{x}_t) + \frac{1}{2} \mathbf{B}_{k-1}^\top \Sigma \Sigma^\top \mathbf{B}_{k-1} - \mathbf{B}_{k-1}^\top (\boldsymbol{\lambda}_0 + \boldsymbol{\lambda}_x \mathbf{x}_t) \\
&= A_{k-1} - r_t + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^\mathbb{P} - \boldsymbol{\lambda}_0) + \frac{1}{2} \mathbf{B}_{k-1}^\top \Sigma \Sigma^\top \mathbf{B}_{k-1} + \mathbf{B}_{k-1}^\top \boldsymbol{\Phi}^\mathbb{P} \mathbf{x}_t - \mathbf{B}_{k-1}^\top \boldsymbol{\lambda}_x \mathbf{x}_t \\
&= A_{k-1} - r_t + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^\mathbb{P} - \boldsymbol{\lambda}_0) + \frac{1}{2} \mathbf{B}_{k-1}^\top \Sigma \Sigma^\top \mathbf{B}_{k-1} + (\mathbf{B}_{k-1}^\top \boldsymbol{\Phi}^\mathbb{P} - \mathbf{B}_{k-1}^\top \boldsymbol{\lambda}_x) \mathbf{x}_t \\
&= A_{k-1} - \alpha - \boldsymbol{\beta}^\top \mathbf{x}_t + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^\mathbb{P} - \boldsymbol{\lambda}_0) + \frac{1}{2} \mathbf{B}_{k-1}^\top \Sigma \Sigma^\top \mathbf{B}_{k-1} + (\mathbf{B}_{k-1}^\top \boldsymbol{\Phi}^\mathbb{P} - \mathbf{B}_{k-1}^\top \boldsymbol{\lambda}_x) \mathbf{x}_t \\
&= A_{k-1} - \alpha + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^\mathbb{P} - \boldsymbol{\lambda}_0) + \frac{1}{2} \mathbf{B}_{k-1}^\top \Sigma \Sigma^\top \mathbf{B}_{k-1} + (\mathbf{B}_{k-1}^\top (\boldsymbol{\Phi}^\mathbb{P} - \boldsymbol{\lambda}_x) - \boldsymbol{\beta}^\top) \mathbf{x}_t.
\end{aligned}$$

From this we can clearly see that A_k is given by

$$A_k = A_{k-1} - \alpha + \mathbf{B}_{k-1}^\top (\boldsymbol{\mu}^\mathbb{P} - \boldsymbol{\lambda}_0) + \frac{1}{2} \mathbf{B}_{k-1}^\top \Sigma \Sigma^\top \mathbf{B}_{k-1},$$

and $\mathbf{B}_k$ is given by

$$\mathbf{B}_k^\top = \mathbf{B}_{k-1}^\top (\boldsymbol{\Phi}^\mathbb{P} - \boldsymbol{\lambda}_x) - \boldsymbol{\beta}^\top.$$

We can further rewrite the expression for $\mathbf{B}_k$ by using the fact that $\mathbf{B}_1^\top = -\boldsymbol{\beta}^\top$. For $k = 1$

$$\mathbf{B}_1^\top = -\boldsymbol{\beta}^\top.$$

For $k = 2$

$$\begin{aligned}
\mathbf{B}_2^\top &= \mathbf{B}_1^\top (\boldsymbol{\Phi}^\mathbb{P} - \boldsymbol{\lambda}_x) - \boldsymbol{\beta}^\top \\
&= -\boldsymbol{\beta}^\top (\boldsymbol{\Phi}^\mathbb{P} - \boldsymbol{\lambda}_x) - \boldsymbol{\beta}^\top \\
&= -\boldsymbol{\beta}^\top \left((\boldsymbol{\Phi}^\mathbb{P} - \boldsymbol{\lambda}_x) + (\boldsymbol{\Phi}^\mathbb{P} - \boldsymbol{\lambda}_x)^0 \right) \\
&= -\boldsymbol{\beta}^\top \left((\boldsymbol{\Phi}^\mathbb{Q})^1 + (\boldsymbol{\Phi}^\mathbb{Q})^0 \right).
\end{aligned}$$

For $k = 3$

$$\begin{aligned}
\mathbf{B}_3^\top &= \mathbf{B}_2^\top (\boldsymbol{\Phi}^\mathbb{P} - \boldsymbol{\lambda}_x) - \boldsymbol{\beta}^\top \\
&= \mathbf{B}_2^\top \boldsymbol{\Phi}^\mathbb{Q} - \boldsymbol{\beta}^\top \\
&= -\boldsymbol{\beta}^\top \left((\boldsymbol{\Phi}^\mathbb{Q})^1 + (\boldsymbol{\Phi}^\mathbb{Q})^0 \right) \boldsymbol{\Phi}^\mathbb{Q} - \boldsymbol{\beta}^\top \\
&= -\boldsymbol{\beta}^\top \left((\boldsymbol{\Phi}^\mathbb{Q})^2 + (\boldsymbol{\Phi}^\mathbb{Q})^1 \right) - \boldsymbol{\beta}^\top \\
&= -\boldsymbol{\beta}^\top \left((\boldsymbol{\Phi}^\mathbb{Q})^2 + (\boldsymbol{\Phi}^\mathbb{Q})^1 + (\boldsymbol{\Phi}^\mathbb{Q})^0 \right).
\end{aligned}$$

We then see that the general pattern for k is

$$\mathbf{B}_k^\top = -\boldsymbol{\beta}^\top \sum_{i=0}^{k-1} (\boldsymbol{\Phi}^\mathbb{Q})^i.$$

Transposed using $(AB)^\top = B^\top A^\top$ we get

$$\mathbf{B}_k = -\sum_{i=0}^{k-1} [(\boldsymbol{\Phi}^\mathbb{Q})^\top]^i \boldsymbol{\beta}.$$

12.8 Appendix B1: Extra Figures

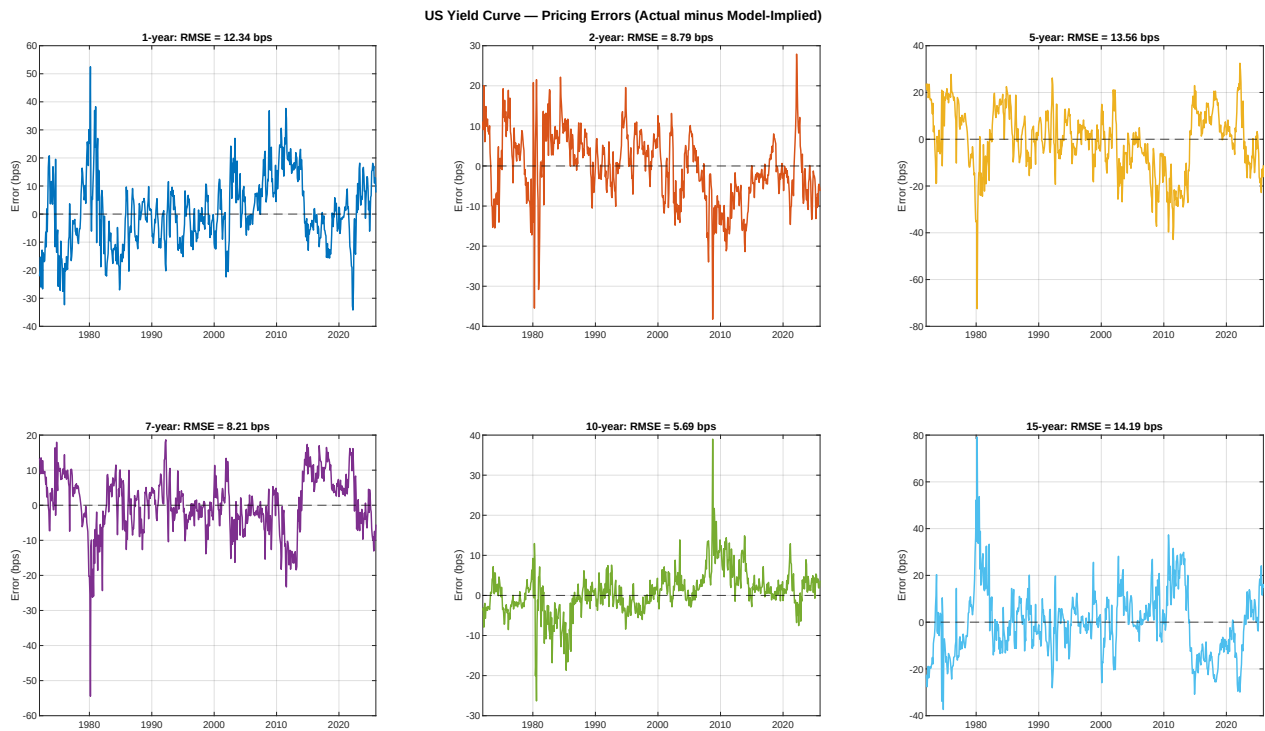


Figure 18: Pricing Errors for the Two-Factor GATSM (CMAES)

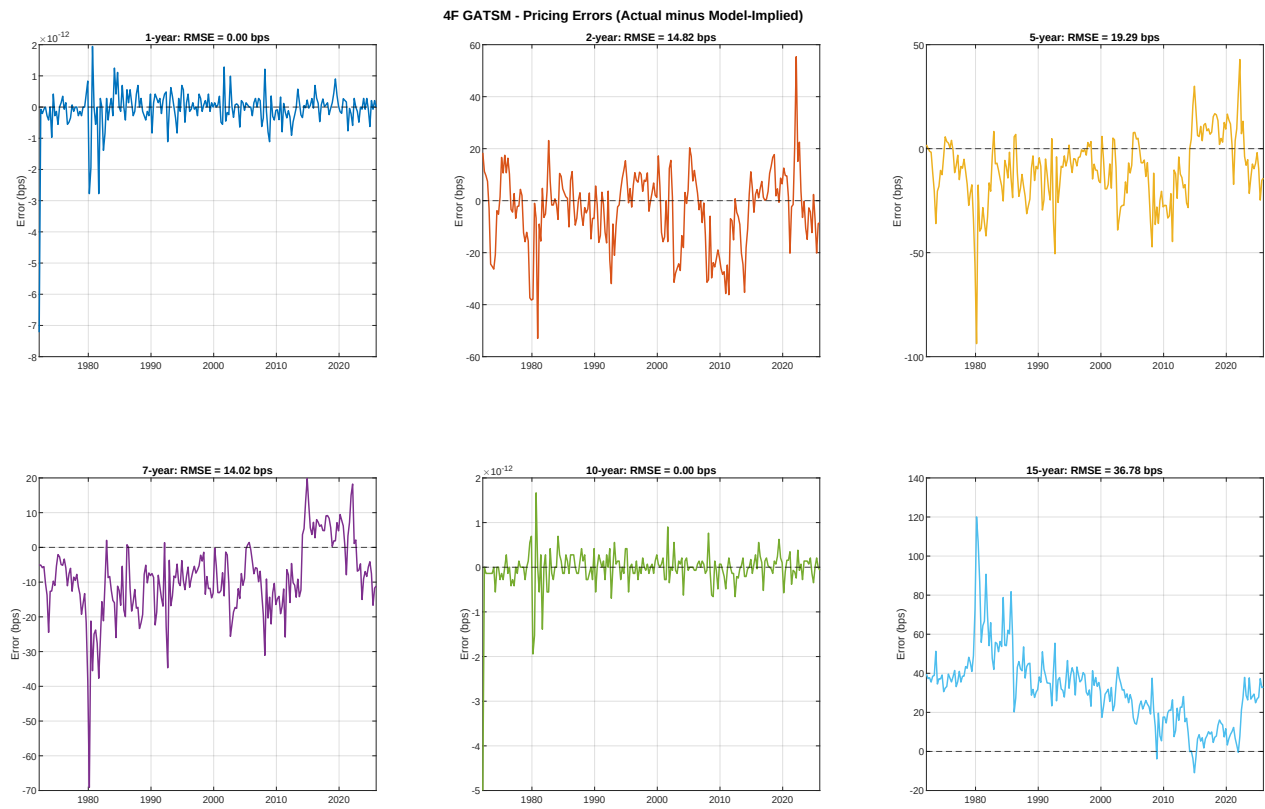


Figure 19: Pricing Errors for the Four-Factor GATSM (NS)

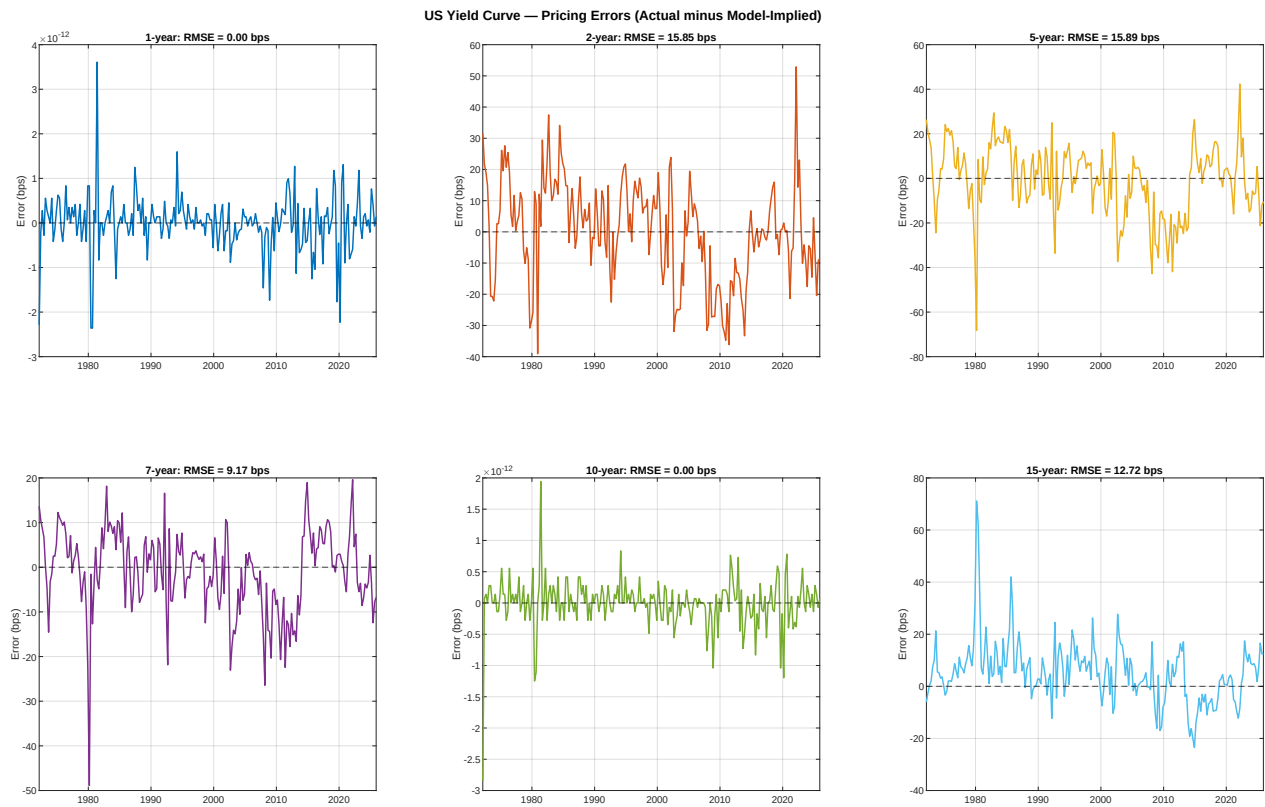


Figure 20: Pricing Errors for the Four-Factor GATSM with surveys (1Q)

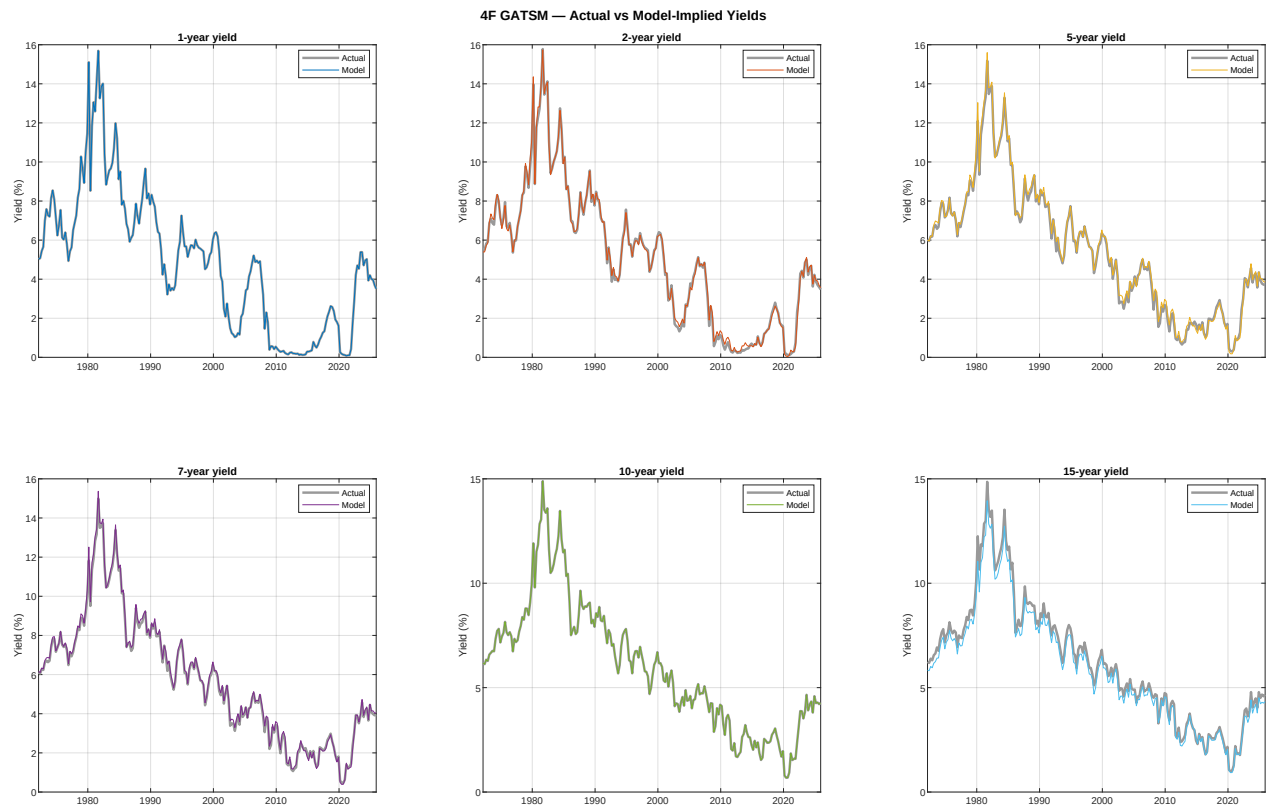


Figure 21: Yield Fit Plot for the Four-Factor GATSM (NS)

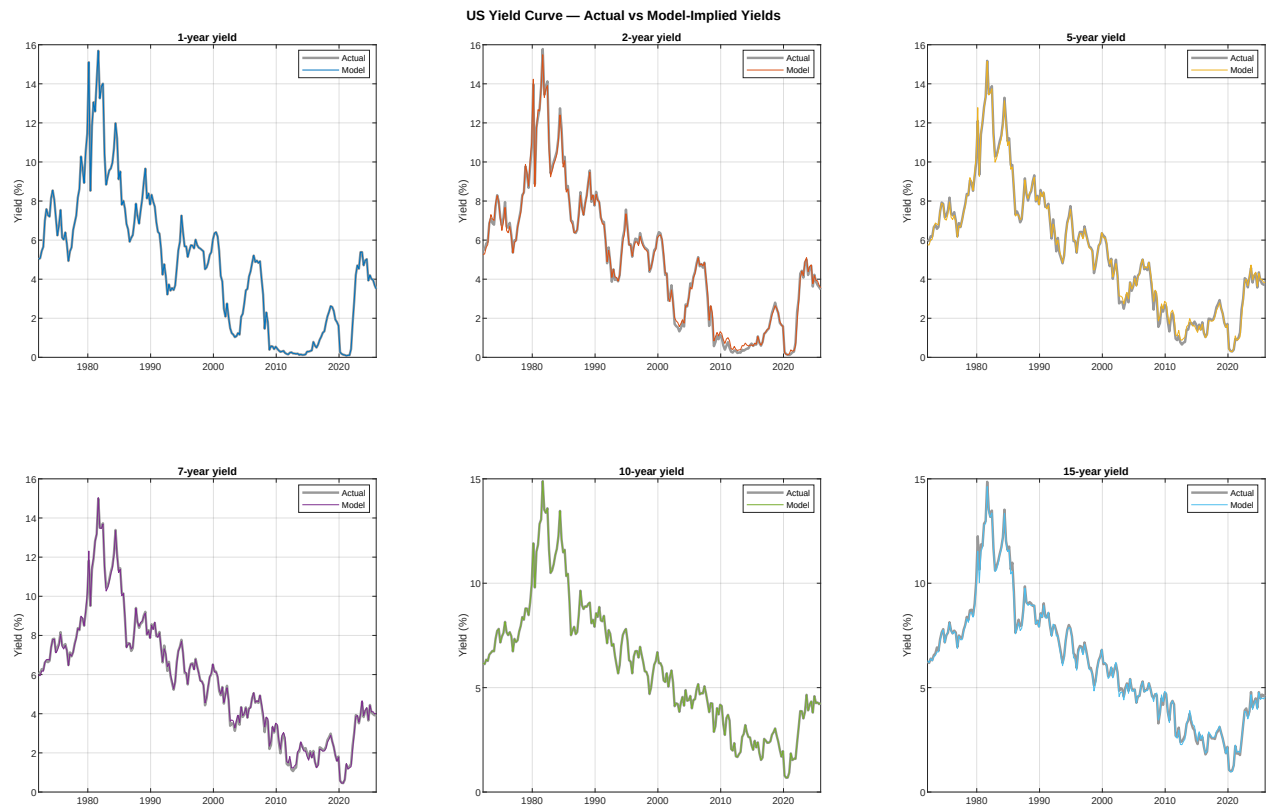


Figure 22: Yield Fit Plot for the Four-Factor GATSM with surveys (1Q)

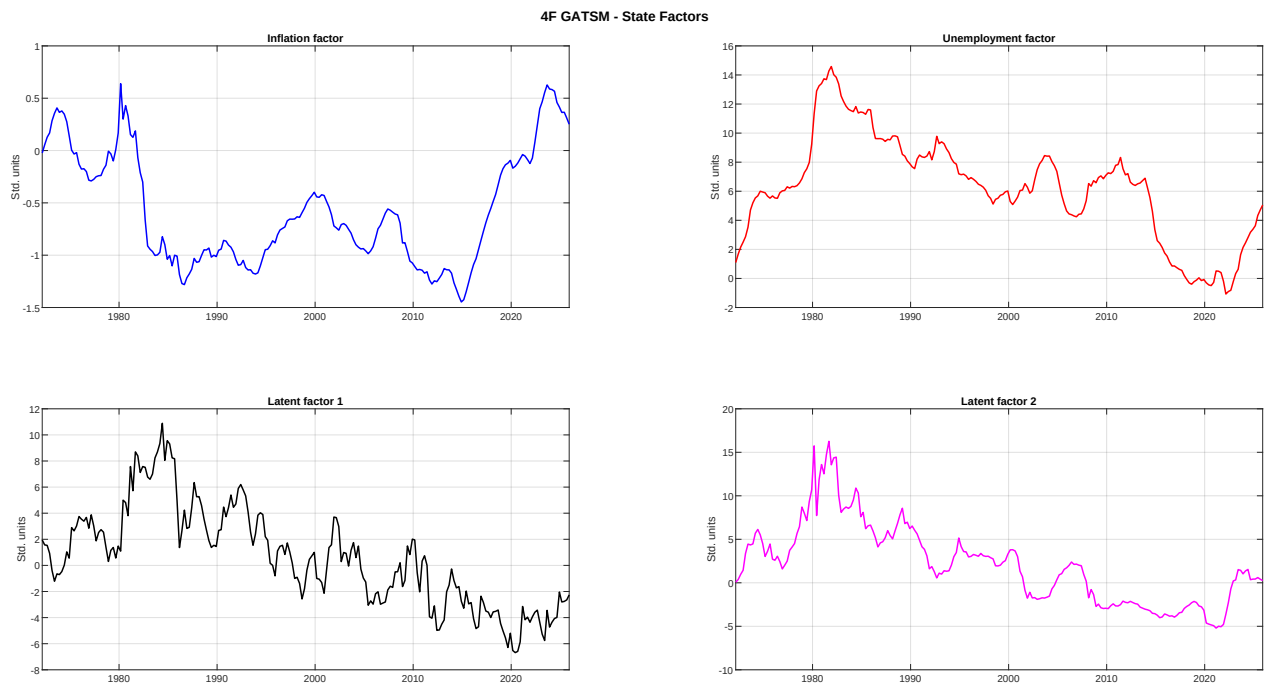


Figure 23: Figure of Factors in the Four-Factor GATSM (NS)

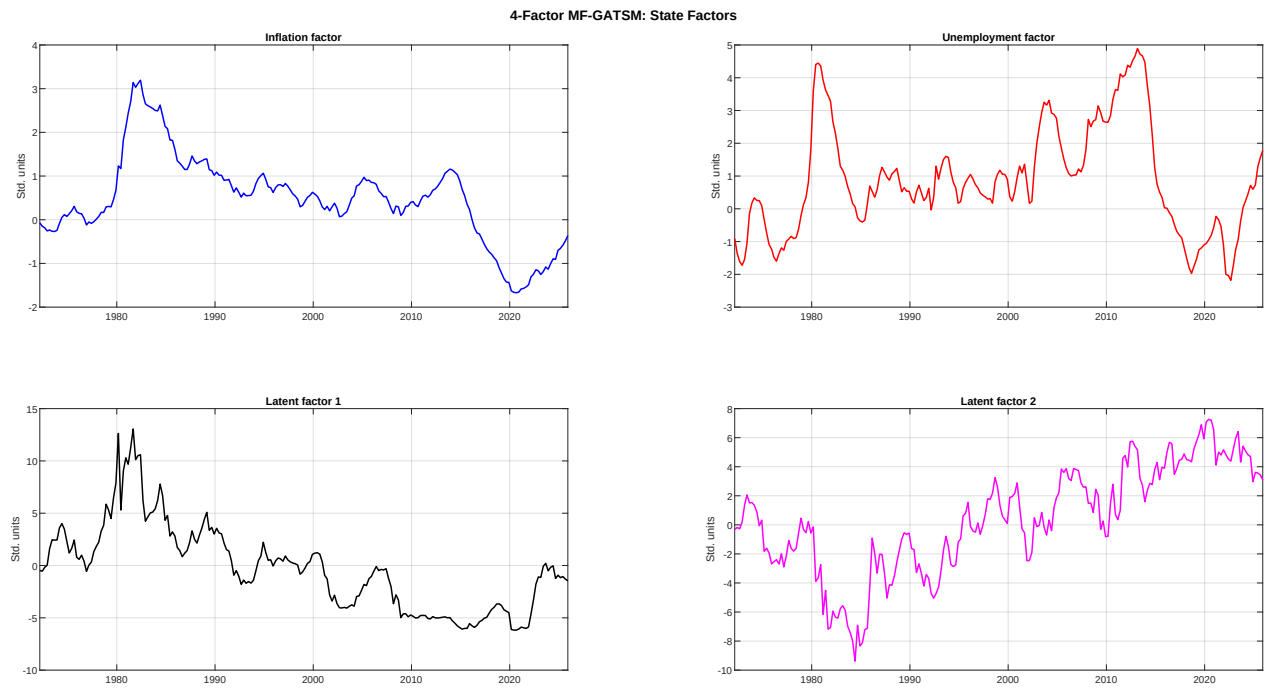


Figure 24: Figure of Factors in the Four-Factor GATSM with surveys (1Q)

12.9 Appendix C1: Main Survey Script

```
% This script incorporates surveys into the four-factor GATSM
%
% Files needed in dir
%   US_monthly_yields_Jan1972_Dec2025.csv
%   US_monthly_yields_Jan1972_Dec2025_maturities.csv
%   US_macro_inflation_unemployment_MF.csv

%% User Settings

% Maturities to use in estimation (years).
% Expressed in quarters; must match values in the maturities
  CSV
% when converted from years (multiply years by 4).
matSelect = [1 2 5 7 10 15] * 4;      % Maturities in quarters

% Yields priced exactly (used to invert latent factors); must
  equal nxL = 2.
% Remaining maturities are priced with measurement error.
% We choose the 1-year and 10-year to span the curve.
exactYears = [1 10];                  % years; must be a subset
  of matSelect/4
errorYears = setdiff(matSelect/4, exactYears);

% Factor dimensions
nxM = 2;      % number of observable macro factors (inflation,
  unemployment)
nxL = 2;      % number of latent yield factors
nx  = nxM + nxL; % total state dimension (= 4)

% Annualisation factor: quarterly data -> annual yields
ann = 4;
```

```

w_svy = 1.0;

%% Load data

% 2a: Yield data (monthly CSV -> quarterly sub-sample)
yields_raw = csvread('Data/US_monthly_yields_Jan1972_Dec2025.
    csv');
mats_years = csvread('Data/
    US_monthly_yields_Jan1972_Dec2025_maturities.csv');

% Validate that every requested maturity exists in the CSV
assert(all(ismember(matSelect/ann, mats_years)), ...
    ['One or more maturities in matSelect are not available in
    the CSV. ' ...
    'Available maturities (years): ' num2str(mats_years)]);

% Sub-sample monthly -> quarterly: rows 3, 6, 9, ... (Mar, Jun,
    Sep, Dec)
% Starting Jan 1972, the first quarterly obs is March 1972 =
    row 3.
T_monthly = size(yields_raw, 1);
qIdx      = 3 : 3 : T_monthly;      % row indices of quarterly
    observations
T          = length(qIdx);          % number of quarterly obs

numObs = length(matSelect);          % total number of yield
    maturities used
data    = zeros(numObs, T);          % numObs x T (annualised
    decimal)
for i = 1:numObs
    col      = find(mats_years == matSelect(i)/ann);
    data(i,:) = yields_raw(qIdx, col)' / 100;
end

```

```

% 2b:
% Y1 has nxL rows      one per latent factor (used to invert
    x_latent).
% Y2 has Ne = numObs - nxL rows      priced with common
    measurement error stdY.
idxExact  = ismember(matSelect/ann, exactYears);    % logical,
    length numObs
idxError  = ~idxExact;
Ne        = sum(idxError);

matSelect_Y1 = matSelect(idxExact);    % nxL-element vector (
    quarters)
matSelect_Y2 = matSelect(idxError);    % Ne-element vector (
    quarters)

Y1 = data(idxExact, :);    % nxL x T  yields priced without
    error
Y2 = data(idxError, :);    % Ne  x T  yields priced with
    measurement error

% 2c: Macro data (quarterly, aligned to yield sub-sample)
% The macro CSV has valid values every 3rd row (same cadence as
    qIdx).
macro_raw = csvread('Data/US_macro_inflation_unemployment_MF.
    csv'); % T_monthly x 2
macro_Q    = macro_raw(qIdx, :);    % T x 2  [inflation,
    unemployment]

% Fill any residual NaNs by linear interpolation
for j = 1:2
    col    = macro_Q(:,j);
    nanIdx = isnan(col);
    if any(nanIdx)

```

```

        col(nanIdx) = interp1(find(~nanIdx), col(~nanIdx), ...
                               find(nanIdx), 'linear', 'extrap')
    ;

    macro_Q(:,j) = col;
end
end

% Normalise to zero mean and unit variance
macro_mean = mean(macro_Q);
macro_std  = std(macro_Q);
xMacro     = ((macro_Q - macro_mean) ./ macro_std)';    % nxM x
T

if w_svy > 0
    fprintf(' Loading KO survey data (w_svy=%.2f)...\\n', w_svy
    );
    T_infl_raw = readtable('US_SurveyData.xlsx', 'Sheet', '
        Inflation', ...
                           'ReadRowNames', true);
    T_unemp_raw = readtable('US_SurveyData.xlsx', 'Sheet', '
        Unemployment', ...
                           'ReadRowNames', true);

    surv_infl_raw = double(T_infl_raw(:, 'dpgdp3'));    % T x
    1 (% ann., 1Q-ahead)
    surv_unemp_raw = double(T_unemp_raw(:, 'UNEMP3'));    % T x
    1 (% , 1Q-ahead)

    % Normalise each survey series by its own mean and std
    svy_mean_infl = mean(surv_infl_raw, 'omitnan');
    svy_std_infl  = std( surv_infl_raw, 'omitnan');
    svy_mean_unemp = mean(surv_unemp_raw, 'omitnan');
    svy_std_unemp  = std( surv_unemp_raw, 'omitnan');

```

```

xSvy_infl = (surv_infl_raw - svy_mean_infl) /
    svy_std_infl;    % T x 1
xSvy_unemp = (surv_unemp_raw - svy_mean_unemp) /
    svy_std_unemp;    % T x 1

% Stack: xSurvey is nxM x T (rows = [infl; unemp])
xSurvey = [xSvy_infl'; xSvy_unemp'];

fprintf('  Survey normalised: infl(mean=%.4f,std=%.4f),
    unemp(mean=%.4f,std=%.4f)\n', ...
    mean(xSvy_infl), std(xSvy_infl), mean(xSvy_unemp), std(
        xSvy_unemp));
else
    xSurvey = [];    % no survey data; Block m OLS uses actual
        only
    fprintf('  w_svy=0: survey augmentation disabled (standard
        V23).\n');
end

dateStart = datenum(1972, 3, 1);
setup_dates = dateStart + (0:T-1)' * 91;    % approximate
    quarterly spacing
dateVec = datetime(1972, 3, 1) + calmonths(3*(0:T-1));

fprintf('  Quarterly obs: T = %d\n', T);
fprintf('  Maturities (years): %s\n', num2str(matSelect/ann));
fprintf('  Priced exactly (Y1): %s yr\n', num2str(exactYears));
fprintf('  Priced with error (Y2): %s yr\n', num2str(errorYears
    ));

%% First-stage OLS (reduced-form VAR)

T1 = T - 1;    % usable observations after one lag

ym_act = xMacro(:, 2:end)';    % T1 x nxM (

```



```

    actual at t+1)
xm_act = [ones(T1,1), xMacro(:,1:end-1)']; % T1 x (1+nxM)

if w_svy > 0 && ~isempty(xSurvey)
    % Survey dependent variable: xSurvey(:,t) = E^svy_{t-1}[x_{macro,t}]
    % We need survey at t matching actual at t (both are t+1 in the OLS).
    % xSurvey(:,t) = dpdgp3/UNEMP3 at t = 1Q-ahead forecast made at t for t+1.
    % For OLS rows t=1..T1:
    %   actual dep = xMacro(:,t+1) indexed as xMacro(:,2:end) -> ym_act
    %   survey dep = xSurvey(:,t) indexed as xSurvey(:,1:end-1) -> ym_svy
    %   regressor = xMacro(:,t) indexed as xMacro(:,1:end-1) -> xm_act/xm_svy
    % Note: dpdgp3_t forecasts x_m(t+1) using info at t, with no look-ahead.
    % For h=1 surveys: dpdgp3_t forecasts x_m(t+1), so for OLS row t
    % we need xSurvey(:,t), i.e. indices 1..T-1 (not 2..T).
    ym_svy = xSurvey(:, 1:end-1)'; % T1 x nxM (survey at t, forecasting t+1)
    xm_svy = [ones(T1,1), xMacro(:,1:end-1)']; % T1 x (1+nxM) (same RHS as actual)

    % Stack with weight w_svy
    ym = [ym_act; w_svy * ym_svy];
    xm = [xm_act; w_svy * xm_svy];
else
    ym = ym_act;
    xm = xm_act;
end

```

```

paramM      = xm \ ym;
Am_star     = paramM(1, :)';                                % nxM x 1
phiP_starmm = paramM(2:end, :)';                            % nxM x
              nxM

% Residuals and Omega_star computed from ACTUAL data only (
  structural noise)
umt_star    = ym_act - xm_act * paramM;                    % T1 x nxM
Omega_star  = (umt_star' * umt_star) / T1;                  % nxM x
              nxM

fprintf('  Block m: %d actual + %d survey obs (w_svy=%.2f).\n',
      ...
      T1, T1 * double(w_svy>0), w_svy);
fprintf('  phiP_starmm:\n');  disp(phiP_starmm);

% Block 1: Y1 on lagged Y1 and contemporaneous macro
y1      = Y1(:, 2:end)';
              % T1 x nxL
x1      = [ones(T1,1), Y1(:,1:end-1)', xMacro(:,2:end)'];
              % T1 x (1+nxL+nxM)
param1   = x1 \ y1;
A1_star  = param1(1, :)';                                % nxL x 1
  (annual)
phiP_star11 = param1(2:1+nxL, :)';                        % nxL x nxL
  (lagged Y1)
psiP_star1m = param1(2+nxL:end, :)';                      % nxL x nxM
  (contemp. macro, annual)
ult_star   = y1 - x1 * param1;                            % T1 x nxL
Omega_star1 = (ult_star' * ult_star) / T1;                 % nxL x nxL
  (annual^2)

% Block 2: Y2 on contemporaneous macro and Y1

```

```

y2                = Y2(:, 2:end)';

                                     % T1 x Ne
x2                = [ones(T1,1), xMacro(:,2:end)', Y1(:,2:end)'];
               % T1 x (1+nxM+nxL)
param2            = x2 \ y2;
A2_star           = param2(1, :)';                                     % Ne x 1 (
               annual)
phiP_star2m       = param2(2:1+nxM, :)';                             % Ne x nxM
phiP_star21       = param2(2+nxM:end, :)';                             % Ne x nxL
u2t_star          = y2 - x2 * param2;                                 % T1 x Ne

fprintf('  OLS complete.\n');

%% Recover sigma and phiP analytically
sigma_macro = chol(Omega_starm, 'lower');                             % nxM x nxM
               lower triangular
sigma        = blkdiag(sigma_macro, eye(nxL)); % nx x nx

B1l_chol     = chol(Omega_star1, 'lower');                             % nxL x nxL (
               Cholesky of Y1 resid var)
phiP_mm      = phiP_starmm;                                             % nxM x nxM (
               from macro OLS)
phiP_ll      = B1l_chol \ phiP_star11 * B1l_chol; % nxL x nxL
               (similarity)
phiP         = blkdiag(phiP_mm, phiP_ll);                             % nx x nx (
               block diagonal)

eigsP = abs(eig(phiP));
fprintf('  max|eig(phiP)| = %.4f ', max(eigsP));
if max(eigsP) >= 1
    fprintf('  WARNING: P-dynamics non-stationary.\n');
else
    fprintf('  (stationary)\n');
end

```

```

fprintf('  sigma_macro =\n');  disp(sigma_macro);

%% Solve for (Lambda_mm, Lambda_ll, beta) numerically

% OLS targets (all in annual decimal units)
B1m_OLS = psiP_star1m; % nxL x nxM
B2m_OLS = phiP_star2m + phiP_star21 * B1m_OLS; % Ne x nxM
B1B1_OLS = Omega_star1; % nxL x nxL
B2B1_OLS = phiP_star21 * Omega_star1; % Ne x nxL

% Compact maturity struct
mature.exact = matSelect_Y1;
mature.error = matSelect_Y2;

% ---- CMA-ES options (shared by both subproblems)
-----
cmaOpts.Display      = 'off';
cmaOpts.Plotting     = 'off';
cmaOpts.Saving       = 0;
cmaOpts.VerboseModulo = 0;
cmaOpts.TolFun       = 1e-14;
cmaOpts.TolX         = 1e-14;
cmaOpts.MaxFunEvals   = 1e6;
cmaOpts.StopOnWarnings = 'no';

% Subproblem A: macro block (Lambda_mm, beta_m)
% Parameter vector: p = [Lambda_mm(:); beta_m] (nxM^2 + nxM
%   elements)
% SSR = ||local_macroBlock_residuals(p)||^2
%
% SIGN: beta_m[k] sign equals the column-sum sign of B1m_OLS.
% SCALE: |beta_m[k]| ~ max(|B1m_OLS[:,k]|) / (ann * 0.86).

beta_m_sign = sign(sum(B1m_OLS, 1)); %

```

```

    1 x nxM
beta_m_scale = max(abs(B1m_OLS), [], 1)' / (ann * 0.86);    %
    nxM x 1

% Lower/upper bounds for beta_m, derived from the sign and
% expected scale.
% For a positive-sign factor: lb = +scale/10, ub = +5*scale (
% positive range)
% For a negative-sign factor: lb = -5*scale, ub = -scale/10 (
% negative range)
% The floor of scale/10 (away from zero) prevents CMA-ES from
% collapsing a
% factor to zero. min/max ensure lb < ub regardless of sign.
lb_A = [-5*ones(nxM^2,1); min(beta_m_sign'.*beta_m_scale/10,
    beta_m_sign'.*beta_m_scale*5)];
ub_A = [ 5*ones(nxM^2,1); max(beta_m_sign'.*beta_m_scale/10,
    beta_m_sign'.*beta_m_scale*5)];

x0_A    = [zeros(nxM^2,1); beta_m_sign'.*beta_m_scale];
sig0_A  = [0.5*ones(nxM^2,1); beta_m_scale];

cmaOpts.LBounds = lb_A;
cmaOpts.UBounds = ub_A;
cmaOpts.PopSize = 4 + floor(3*log(length(x0_A)));

objA = @(p) sum(local_macroBlock_residuals(p, phiP_mm,
    sigma_macro, ...
                                B1m_OLS, B2m_OLS,
                                mature, nxM, ann)
                                .^2);
[xBestA, fBestA] = cmaes_dsgeDisplay(objA, x0_A, 1, sig0_A,
    cmaOpts);

Lambda_mm = reshape(xBestA(1:nxM^2), nxM, nxM);

```

```

beta_m      = xBestA(nxM^2+1 : end);
phiQ_mm     = phiP_mm - sigma_macro * Lambda_mm;
fprintf('  Macro CMA-ES SSR: %.2e\n', fBestA);
fprintf('  eig(phiQ_mm): %s\n', num2str(abs(eig(phiQ_mm))'));
fprintf('  beta_m: %s\n', num2str(beta_m'));

% Subproblem B: latent block (Lambda_ll_Jordan, beta_l)
% Parameter vector: p = [lev; lb_off; lc_off; beta_l]    (3 +
%   nxL elements)
% Lambda_ll = [[lev, lb_off], [lc_off, lev]]    equal-diagonal
%   Jordan form
% phiQ_ll    = phiP_ll - Lambda_ll
%
% IDENTIFICATION (H-W Propositions 1-2):
%   (a) H-W Proposition 2: for Nl=2, the Jordan
%       parameterisation has
%       two observationally equivalent solutions related by
%       transposing
%       phiQ_ll. Resolved by the canonical condition lb <= lc (
%       i.e. the
%       (1,2) off-diagonal of Lambda_ll <= its (2,1)
%       counterpart).
%   (b) Sign normalisation (H-W Proposition 1): beta_l[k] > 0
%       for all
%       latent factors. Flipping sign of beta_l[k] and of row/
%       col k of
%       phiQ_ll leaves B loadings unchanged    this is the
%       remaining
%       rotation ambiguity for each factor independently.
%       Resolved by
%       enforcing strict positivity via CMA-ES lower bounds.
%
% SCALE: |beta_l[k]| ~ diag(B11_chol)[k] / (ann * 0.86)

```

```

beta_l_scale = diag(B1l_chol) / (ann * 0.86);    % nxL x 1

% Strictly positive lower bound for beta_l (H-W sign
  normalisation).
% BETA_L FLOOR: 0.0001 on BOTH entries.
% This blocks the degenerate near-zero basin (beta_l~0) that
  CMA-ES
% The Jordan swap can permute the two entries, so both must be
  floored to be safe.
lb_B = [-5; -5; -5; 0.0001; 0.0001];
ub_B = [ 5;  5;  5; beta_l_scale(1)*5;  beta_l_scale(2)*5];

% THREE STARTS for robustness, where we keep the lowest SSR
  solution.
% Subproblem B inputs (B1B1_OLS, B2B1_OLS, phiP_ll) are
  identical to NS
% regardless of w_svy, so the V23 solution is always a valid
  starting point.
%
%   Start 1: canonical form      (b<c satisfied)
%   Start 2: pre-swap form       (b>c; same solution, different
    orientation)
%   Start 3: scale-based default (broad search fallback)
x0_B_s1  = [-0.0689; 0.1351; 0.1520; 0.00226; 0.00062];
sig0_B_s1 = [0.03;    0.03;    0.03;    0.00050; 0.00015];

x0_B_s2  = [-0.0689; 0.1714; 0.1157; 0.00232; 0.00038];
sig0_B_s2 = [0.03;    0.03;    0.03;    0.00050; 0.00010];

x0_B_s3  = [0; 0; 0; beta_l_scale];
sig0_B_s3 = [0.3; 0.1; 0.1; beta_l_scale];

cmaOpts.LBounds = lb_B;
cmaOpts.UBounds = ub_B;

```

```

cmaOpts.PopSize = 4 + floor(3*log(length(x0_B_s1)));

objB = @(p) sum(local_latentBlock_residuals(p, phiP_ll,
      B1B1_OLS, B2B1_OLS, ...
      mature, nxL, ann)
      .^2);

[xB_s1, fB_s1] = cmaes_dsgeDisplay(objB, x0_B_s1, 1, sig0_B_s1,
      cmaOpts);
[xB_s2, fB_s2] = cmaes_dsgeDisplay(objB, x0_B_s2, 1, sig0_B_s2,
      cmaOpts);
[xB_s3, fB_s3] = cmaes_dsgeDisplay(objB, x0_B_s3, 1, sig0_B_s3,
      cmaOpts);

fprintf('  Latent SSR: s1=%.2e  s2=%.2e  s3=%.2e\n', fB_s1,
      fB_s2, fB_s3);

[fBestB, iBest] = min([fB_s1, fB_s2, fB_s3]);
xBestBs = {xB_s1, xB_s2, xB_s3};
xBestB = xBestBs{iBest};
fprintf('  Latent best: start %d (SSR=%.2e)\n', iBest, fBestB);

lev      = xBestB(1);
lb       = xBestB(2);
lc       = xBestB(3);
beta_l   = xBestB(4:5);

Lambda_ll = [lev, lb; lc, lev];
phiQ_ll   = phiP_ll - Lambda_ll;

% Enforce H-W Proposition 2 canonical form: phiQ_ll[1,2] <=
      phiQ_ll[2,1]
%
% the canonical condition is on phiQ_ll itself, NOT on the

```



```

% Lambda_ll entries lb and lc. Since phiQ_ll = phiP_ll -
    Lambda_ll, and
% phiP_ll has non-zero off-diagonals, lb <= lc does NOT imply
% phiQ_ll[1,2] <= phiQ_ll[2,1] in general.
%
% The two equivalent solutions are phiQ_ll and phiQ_ll' (
    transpose),
% with beta_l permuted accordingly. We keep the representative
% with phiQ_ll[1,2] <= phiQ_ll[2,1].
if phiQ_ll(1,2) > phiQ_ll(2,1)
    phiQ_ll = phiQ_ll';          % transpose gives the other
    equivalent form
    beta_l = beta_l([2,1]);      % permute beta_l to match
    % Recompute Lambda_ll from the canonical phiQ_ll
    Lambda_ll = phiP_ll - phiQ_ll;
end
fprintf('  Latent CMA-ES SSR: %.2e\n', fBestB);
fprintf('  eig(phiQ_ll): %s\n', num2str(abs(eig(phiQ_ll))));
fprintf('  beta_l: %s (both > 0: %d)\n', num2str(beta_l'), all
    (beta_l > 0));
fprintf('  Jordan b<=c: b=%.4f, c=%.4f (satisfied: %d)\n', ...
    phiQ_ll(1,2), phiQ_ll(2,1), phiQ_ll(1,2) <= phiQ_ll
    (2,1) + 1e-10);

% Assemble full model matrices (block-diagonal: Q-independence)
phiQ = blkdiag(phiQ_mm, phiQ_ll);
beta = [beta_m; beta_l];

eigsQ = abs(eig(phiQ));
fprintf('  eig(phiQ) (moduli): %s\n', num2str(sort(eigsQ, '
    descend')));
fprintf('  beta = %s\n', num2str(beta));

%% Compute B loadings for downstream steps

```

```

[B1m_ann, ~, B1l_ann, B2l_ann] = local_computeB_ann(phiQ, beta,
    mature, nxM, nxL, ann);

%% Solve for (muQ, alpha) numerically

% Solve:  A2_star - A2_ann(alpha,muQ) + B2l_ann / B1l_ann *
    A1_ann = 0
% for x = [alpha; muQ_macro]    (muQ_latent = 0 by normalisation
    )

muQ = zeros(nx, 1);    % muQ_latent = 0 by normalisation;
    muQ_macro solved below

% Parameter vector: x = [alpha; muQ_macro]    (1 + nxM elements)
%
% ALPHA BOUNDS: narrow to economically plausible range.
%   H-W delta0 (alpha) = -0.0082 quarterly which is around r0 =
%   -3.3% ann?
%   The unconditional short rate is  $E[r] = \text{ann} * (\alpha + \beta' * (I - \phi P)^{-1} * \mu P)$ .
%   With  $E[y_{1yr}]$  4.86%, alpha should be in roughly -0.02
%   to +0.025 quarterly.
%   We use [-0.03, +0.03] to give CMA-ES adequate room without
%   allowing
%   alpha to go to -0.05 which results in degenerate solution
%   that occurs with wider bounds.
%
% UNCONDITIONAL MEAN SOFT CONSTRAINT:
%   The A-intercept moment conditions alone do not uniquely pin
%   down the
%   (alpha, muQ) decomposition: many combinations give the same
%   A2_star fit
%   but produce wildly different unconditional yield levels (we

```

```

    observed
%   alpha = -0.05, E[r] = -7.3% when the data mean is +4.9%).
%   We add a soft penalty that anchors the unconditional short
    rate:
%       penalty = w * (ann*(alpha + beta'*(I-phiP)^{-1}*muP) -
    mean_y1yr)^2
%   where w is chosen so the penalty is comparable to the
    A2_star residuals.
%   This does NOT change what is being estimated (it is not a
    hard constraint)
%   but prevents CMA-ES from settling on degenerate (alpha, muQ
    ) pairs.

mean_y1yr = mean(data(1,:));           % mean 1yr yield (
    annual decimal)

x0_7      = [mean_y1yr/ann; zeros(nxM,1)]; % alpha start: mean
    short rate / ann
sig0_7    = [0.005; 0.3*ones(nxM,1)];

% TIGHTENED muQ BOUNDS (from Survey experience):
%   muQ = c^Q: the constant market price of risk intercept.

%   Allowing 50 lets Step 7 find
%   large-muQ solutions that imply huge muP_1.
%   Bounding at 3 prevents this while giving 30 the HW
    magnitude.
lb_7      = [-0.03; -3*ones(nxM,1)];
ub_7      = [ 0.03;  3*ones(nxM,1)];

cmaOpts7      = cmaOpts;
cmaOpts7.LBounds = lb_7;
cmaOpts7.UBounds = ub_7;
cmaOpts7.PopSize = 4 + floor(3*log(length(x0_7)));

```

```

cmaOpts7.TolFun    = 1e-10;
cmaOpts7.TolX      = 1e-10;

% we have three soft constraints
%
% Constraint 1      alpha anchor (existing):
%   Penalise alpha deviating from mean_y1yr/ann.
%   Prevents alpha drifting to boundaries while muQ compensates
%   .
%
% Constraint 2      muQ shrinkage (from Survey V7):
%   Penalise large |muQ_macro|.  muQ enters A_ann through the
%   recursion;
%   large muQ -> large A_ann -> large muP_1 -> large E[x_latent
%   ] -> level wrong.
%   Even with tight bounds, shrinkage prevents muQ from sitting
%   at 3 .
%
% Constraint 3      FULL UNCONDITIONAL YIELD LEVEL:
%   Embed the Step 8 muP_1 recovery inside the objective so
%   that
%    $E[r] = \text{ann} * (\alpha + \beta * (1 - \phi_P)^{-1} * \mu_P)$  can be
%   directly penalised.
%   This breaks the circular dependency between Step 7 and Step
%   8 by
%   computing the implied muP_1 forward from any candidate (
%   alpha, muQ).
%   We anchor at the 10yr yield mean (data mean = 5.97%)
%   because the 1yr
%   is already anchored by Constraint 1, and the 10yr level
%   captures the
%   long-end slope problem most directly.
%
%   Implementation: for each candidate x=[alpha;muQ], compute

```

```

    A1_ann(x),
%   recover muP_l analytically, then compute E[r] and E[y_10yr]
    and penalise.
%
%   A2 residuals scaled by 1e6  -> w_mean and w_muQ in same
    units
%   Yield level in decimal      -> w_level scales to match 1e6
    -scaled residuals

w_mean  = 1e6;      % alpha anchor weight
w_muQ   = 1e4;      % muQ shrinkage weight
w_level = 5e5;      % level anchor weight (0.01% deviation = 1e-4
    decimal * 5e5 = 50 cost)

IminusPhiP_ll = eye(nxL) - phiP_ll;    % precompute for speed

% muP_m = Am_star (AP2003 independence: macro intercept = Block
    m OLS intercept).
% Defined here so it is available inside the step7_objective
    closure;
% Step 8 will also assign it explicitly for clarity.
muP_m = Am_star;

obj7 = @(x) step7_objective(x, mature, phiQ, beta, sigma, ...
    A2_star, A1_star, B1l_ann,
    B2l_ann, B1l_chol, ...
    phiP_ll, IminusPhiP_ll, muP_m, ...
    nxM, nxL, ann, mean_y1yr, ...
    w_mean, w_muQ, w_level);

[xBest7, fBest7] = cmaes_dsgeDisplay(obj7, x0_7, 0.5, sig0_7,
    cma0pts7);

alpha      = xBest7(1);

```

```

muQ(1:nxM) = xBest7(2:end);

fprintf('  CMA-ES SSR: %.2e\n', fBest7);
fprintf('  alpha = %.6f  (quarterly; r0 = %.4f%%)\n', alpha,
        alpha*ann*100);
fprintf('  muQ_macro = %s\n', num2str(muQ(1:nxM)'));

%% Recover muP analytically

% With the AP2003 independence assumption (phiP_m1 = phiP_lm =
%   0):
%
%   muP_m = Am_star    (macro intercept from block m OLS; note
%   xMacro has
%
%               zero mean by construction so Am_star
%   0)
%   muP_l = B1l_ann \ (A1_star - A1_ann + B1l_ann * phiP_ll /
%   B1l_ann * A1_ann)
%   muP     = [muP_m; muP_l]

[A1_ann, ~] = local_computeA_ann(alpha, muQ, phiQ, beta, sigma,
    mature, nxM, nxL, ann);

% muP_m = Am_star (independence: zero cross-block correction)
% muP_l uses B1l_chol (same rotation as phiP_ll, so frames are
%   consistent)
muP_m = Am_star;
muP_l = B1l_chol \ (A1_star - A1_ann + B1l_chol * phiP_ll /
    B1l_chol * A1_ann);
muP     = [muP_m; muP_l];

fprintf('  muP = %s\n', num2str(muP));

%% Compute bond pricing loadings (g0, gx, A, B)

```

```

% Bond pricing recursion (quarterly periods)
%
%   k=1:  A(1,1) = -alpha,      B(:,1) = -beta
%   k>1:  A(1,k) = -alpha + A(1,k-1) + B(:,k-1)'*muQ
%           + 0.5*B(:,k-1)'*sigma2*B(:,k-1)
%           B(:,k) = -beta + phiQ'*B(:,k-1)
%
% Here A and B store raw (per-quarter, unnormalised) recursion
% coefficients.
% Annualised yield loadings at maturity n quarters:
%   g0(n) = ann * (-A(1,n) / n)
%   gx(n) = ann * (-B(:,n)' / n)

maxMat = max(matSelect);
A       = zeros(1, maxMat);      % 1 x maxMat (matches 2F
convention)
B       = zeros(nx, maxMat);     % nx x maxMat

sigma2 = sigma * sigma';        % nx x nx innovation covariance

for k = 1:maxMat
    if k == 1
        A(1,k) = -alpha;
        B(:,k) = -beta;
    else
        A(1,k) = -alpha + A(1,k-1) + B(:,k-1)'*muQ + ...
            0.5*B(:,k-1)'*sigma2*B(:,k-1);
        B(:,k) = -beta + phiQ'*B(:,k-1);
    end
end

% Annualised yield loadings at the selected maturities
% Matches 2F convention:  g0 (numObs x 1),  gx (numObs x nx)

```

```

g0 = ann * (-A(1, matSelect)' ./ matSelect');
           % numObs x 1
gx = ann * (-B(:, matSelect)' ./ repmat(matSelect', 1, nx));
           % numObs x nx

% Short-rate loadings (annualised)      used in yield
decomposition
r0 = ann * alpha;
rx = ann * beta';    % 1 x nx

% Pack model struct
model.g0      = g0;
model.gx      = gx;
model.muP     = muP;
model.phiP    = phiP;
model.alpha   = alpha;
model.beta    = beta;
model.phiQ    = phiQ;
model.muQ     = muQ;
model.sigma   = sigma;
model.A       = A;
model.B       = B;
model.r0      = r0;
model.rx      = rx;
model.matSelect = matSelect;
model.nxM     = nxM;
model.nxL     = nxL;

%% Kalman filter

% Measurement equation:  data_t = g0 + gx * x_t + stdY * v_t
%   Y1 rows: measurement error = 0 (priced exactly)
%   Y2 rows: common scalar measurement error stdY

```



```

%
% Transition:   $x_{t+1} = \mu_P + \phi_P * x_t + \sigma * u_{t+1}$ 

stdY = mean(std(u2t_star));    % scalar common measurement error
    std

% Measurement noise covariance Rv (numObs x numObs):
%   zero for Y1 rows, stdY^2 * I for Y2 rows
Rv = zeros(numObs, numObs);
Rv(idxError, idxError) = eye(Ne) * stdY^2;

outKF = local_KalmanFilter(data, g0, gx, Rv, muP, phiP, sigma2)
    ;
outKF.model = model;

fprintf('  Average log-likelihood: %.6f\n', outKF.sumLogL / T);

%% Local Functions

function F = local_macroBlock_residuals(para, phiP_mm,
    sigma_macro, ...
                                     B1m_OLS, B2m_OLS,
                                     mature, nxM, ann)
% Residuals for the macro sub-problem in MCSE Step 2.
% para = [Lambda_mm(:); beta_m]  (nxM^2 + nxM elements)
% phiQ_mm = phiP_mm - sigma_macro * Lambda_mm
Lambda_mm = reshape(para(1:nxM^2), nxM, nxM);
beta_m     = para(nxM^2+1 : end);

phiQ_mm = phiP_mm - sigma_macro * Lambda_mm;
% No stationarity constraint on phiQ_mm. The Q-measure dynamics
% can legitimately have |eig(phiQ_mm)| > 1
% The B-loading recursion stays finite for

```

```

% finite maturities regardless of Q-measure stationarity.

% B macro columns only depend on phiQ_mm and beta_m (Q-
independence)
mats = [mature.exact, mature.error];
B_mac = zeros(length(mats), nxM);
phiQt = phiQ_mm';
for im = 1:length(mats)
    n = mats(im); S = zeros(nxM,1); P = eye(nxM);
    for j = 0:n-1; S = S + P*beta_m; P = P*phiQt; end
    B_mac(im,:) = (ann * S / n)';
end

% if B loadings diverged (phiQ_mm very explosive), return large
penalty
if ~all(isfinite(B_mac(:)))
    nEq = numel(B1m_OLS) + numel(B2m_OLS);
    F = 1e6 * ones(nEq, 1);
    return;
end

nE = length(mature.exact);
B1m_s = B_mac(1:nE, :);
B2m_s = B_mac(nE+1:end, :);

F = [(B1m_OLS - B1m_s) * 1e5; (B2m_OLS - B2m_s) * 1e5];
F = F(:);
end

function F = local_latentBlock_residuals(para, phiP_ll,
    B1B1_OLS, B2B1_OLS, ...
    mature, nxL, ann)
% Residuals for the latent sub-problem in MCSE Step 2.
% para = [lev; lb; lc; beta_1] (3 + nxL elements)
% Lambda_ll = [[lev, lb], [lc, lev]] (equal-diagonal Jordan

```

```

    form)
% phiQ_ll    = phiP_ll - Lambda_ll
lev = para(1); lb = para(2); lc = para(3);
beta_l = para(4:end);

Lambda_ll = [lev, lb; lc, lev];
phiQ_ll    = phiP_ll - Lambda_ll;

% HYBRID CONSTRAINT
%   Latent Q-block: enforce stationarity |eig(phiQ_ll)| <
    0.9999.

%   Without this constraint the landscape has degenerate local
    minima where
%   phiQ_ll is explosive and beta_l[2] collapses to zero,
    making the second
%   latent factor unidentified. The stationarity penalty
    prevents this.
if any(abs(eig(phiQ_ll))) >= 0.9999)
    nEq = nxL*(nxL+1)/2 + numel(B2B1_OLS);
    F = 1e8 * ones(nEq, 1);
    return;
end

mats = [mature.exact, mature.error];
B_lat = zeros(length(mats), nxL);
phiQt = phiQ_ll';
for im = 1:length(mats)
    n = mats(im); S = zeros(nxL,1); P = eye(nxL);
    for j = 0:n-1; S = S + P*beta_l; P = P*phiQt; end
    B_lat(im,:) = (ann * S / n)';
end

% if B loadings diverged (phiQ_ll very explosive) return large

```

```

    penalty
if ~all(isfinite(B_lat(:)))
    nEq = nxL*(nxL+1)/2 + numel(B2B1_OLS);
    F = 1e8 * ones(nEq, 1);
    return;
end
nE      = length(mature.exact);
B1l_s   = B_lat(1:nE, :);
B2l_s   = B_lat(nE+1:end, :);

% Symmetric part of B1l*B1l'
M1 = B1B1_OLS - B1l_s * B1l_s';
r1 = M1([1,2,4]) * 1e5;      % [M(1,1) M(2,1) M(2,2)] = upper-
    left, lower-left, lower-right

M2 = B2B1_OLS - B2l_s * B1l_s';
r2 = M2(:) * 1e5;

F = [r1(:); r2(:)];
end

function [B1m, B2m, B1l, B2l] = local_computeB_ann(phiQ, beta,
    mature, nxM, nxL, ann)
% Compute ANNUALISED B yield loadings for the MCSE moment
    conditions.
%
% Annual yield loading:  gx(n) = ann * (1/n)*sum_{j=0}^{n-1}(
    phiQ')^j * beta
%
% This returns B in annual decimal units, matching the OLS
    targets
% psiP_star1m and Omega_star1 which come from regressions on
    annual yields.
%
```

```

% Returns B partitioned into macro (m) and latent (l) column
    blocks,
% for maturities in mature.exact (Y1) and mature.error (Y2).

nx    = nxM + nxL;
mats = [mature.exact, mature.error];

B_all = zeros(length(mats), nx);
phiQt = phiQ';
for im = 1:length(mats)
    n    = mats(im);
    S    = zeros(nx, 1);
    pow = eye(nx);
    for j = 0:n-1
        S    = S + pow * beta;
        pow = pow * phiQt;
    end
    % Multiply by ann to convert per-quarter to annual units
    B_all(im,:) = (ann * S / n)';
end

nE = length(mature.exact);
B1 = B_all(1:nE, :);
B2 = B_all(nE+1:end, :);

B1m = B1(:, 1:nxM);
B1l = B1(:, nxM+1:end);
B2m = B2(:, 1:nxM);
B2l = B2(:, nxM+1:end);
end

function [A1, A2] = local_computeA_ann(alpha, muQ, phiQ, beta,
    sigma, ...
                                     mature, nxM, nxL, ann)

```

```

% Compute ANNUALISED A yield intercepts.
%
% Annual intercept:  $g_0(n) = \text{ann} * (-A\_bar(n)/n)$ 

sigma2 = sigma * sigma';
nMax    = max([mature.exact, mature.error]);

% Per-quarter recursion (same sign convention as main script
    Step 9)
A_raw    = zeros(1, nMax);
A_bar    = -alpha;
B_bar    = -beta;
A_raw(1) = -A_bar;    % = alpha

for k = 2:nMax
    A_bar    = -alpha + A_bar + B_bar'*muQ + 0.5*(B_bar'*sigma2
        *B_bar);
    B_bar    = phiQ' * B_bar - beta;
    A_raw(k) = -A_bar / k;
end

% Annualise
A_ann = ann * A_raw;

A1 = A_ann(mature.exact)';    % nxL x 1
A2 = A_ann(mature.error)';    % Ne x 1
end

function F = local_alphaMuQ_residuals(x, mature, phiQ, beta,
    sigma, ...
                                     A2_star, B1l_ann,
                                     B2l_ann, nxM, nxL, ann
                                     )
% Residuals of MCSE Step 4:  $A2\_star - A2\_ann + B2l\_ann/B1l\_ann$ 

```

```

    * A1_ann = 0
% All quantities in ANNUAL decimal units.
nx      = nxM + nxL;
alpha   = x(1);
muQ     = zeros(nx, 1);
muQ(1:nxM) = x(2:end);
try
    [A1_ann, A2_ann] = local_computeA_ann(alpha, muQ, phiQ,
        beta, sigma, ...
        mature, nxM, nxL,
        ann);
    F = (A2_star - A2_ann + B2l_ann / B1l_ann * A1_ann) * 1e6;
catch
    F = 1e10 * ones(length(mature.error), 1);
end
end

function out = local_KalmanFilter(y, g0, gx, Rv, h0, hx, Repts)
% Standard Kalman filter
% Measurement: y_t      = g0 + gx*x_t      + Sv*v_t
% Transition:  x_{t+1} = h0 + hx*x_t + Seps*w_{t+1}

[ny, T] = size(y);
nx      = size(hx, 1);

x0      = (eye(nx) - hx) \ h0;
vecP    = (eye(nx^2) - kron(hx, hx)) \ reshape(Repts, nx^2, 1);
P0      = reshape(vecP, nx, nx);

xHat     = zeros(nx, T);
xBar     = zeros(nx, T);
PHat     = zeros(nx, nx, T);
SxHat    = zeros(nx, nx, T);
PBar     = zeros(nx, nx, T);

```

```

VaryBar    = zeros(ny, ny, T);
yBar       = zeros(ny, T);
logL       = zeros(T, 1);
K          = zeros(nx, ny, T);
predError  = zeros(ny, T);

for t = 1:T
    if t == 1
        xBar(:,t)    = h0 + hx*x0;
        PBar(:, :, t) = hx*P0*hx' + Repr;
    else
        xBar(:,t)    = h0 + hx*xHat(:,t-1);
        PBar(:, :, t) = hx*PHat(:, :, t-1)*hx' + Repr;
    end
    yBar(:,t)        = g0 + gx*xBar(:,t);
    VaryBar(:, :, t) = gx*PBar(:, :, t)*gx' + Rv;

    selectY    = ~isnan(y(:,t));
    ny_t       = sum(selectY);
    invVaryBar = VaryBar(selectY, selectY, t) \ eye(ny_t, ny_t)
        ;

    K(:, selectY, t) = PBar(:, :, t) * gx(selectY, :)' *
        invVaryBar;
    predError(:,t)   = y(:,t) - yBar(:,t);
    xHat(:,t)        = xBar(:,t) + K(:,selectY,t) * predError(
        selectY,t);
    PHat(:, :, t)    = PBar(:, :, t) - ...
        K(:,selectY,t)*VaryBar(selectY,selectY,
            t)*K(:,selectY,t)';

    % Ensure PHat stays symmetric and numerically positive
    % definite.
    % Symmetrise first to eliminate floating-point asymmetry.

```



```

PHat(:,:,t) = (PHat(:,:,t) + PHat(:,:,t)') / 2;
% chol with two outputs returns a *partial* factor (p x p)
% when the
% matrix is not PD, which would cause a size mismatch on
% assignment.
[Ltmp, info] = chol(PHat(:,:,t), 'lower');
if info == 0
    SxHat(:,:,t) = Ltmp;
else
    % PHat is not PD: add a ridge until it is
    ridge = 1e-12;
    for iTry = 1:20
        [Ltmp, info] = chol(PHat(:,:,t) + ridge*eye(nx), '
            lower');
        if info == 0
            SxHat(:,:,t) = Ltmp;
            break;
        end
        ridge = ridge * 100;
    end
end

logL(t,1) = -(ny_t/2)*log(2*pi) ...
    - 0.5*log(det(VaryBar(selectY,selectY,t))) ...
    - 0.5*predError(selectY,t)'*invVaryBar*
        predError(selectY,t);
end

sumLogL = sum(logL, 1);

out.data      = y;
out.xHat      = xHat;
out.PHat      = PHat;
out.Sx        = SxHat;
out.logL      = logL;

```

```
out.sumLogL    = sumLogL;
out.Kgain      = K;
out.predError  = predError;
out.VaryBar    = VaryBar;
out.xBar       = xBar;
out.PBar       = PBar;
out.g0         = g0;
out.gx         = gx;
out.h0         = h0;
out.hx         = hx;
out.Rv         = Rv;
out.Reps       = Reps;
out.yBar       = yBar;
end

function res = local_yieldCurveDecom(outKF, model)
% Decomposes model-implied yields into expected short rates and
% term premia.

xhat    = outKF.xHat;
[nx,T]  = size(xhat);
muP     = model.muP;
phiP    = model.phiP;
maxMat  = max(model.matSelect);

xExp = nan(nx, maxMat, T);
for t = 1:T
    for i = 1:maxMat
        if i == 1
            xExp(:,1,t) = xhat(:,t);
        else
            xExp(:,i,t) = muP + phiP * xExp(:,i-1,t);
        end
    end
end
```

```

end

rExp = nan(maxMat, T);
for t = 1:T
    for i = 1:maxMat
        rExp(i,t) = model.r0 + model.rx * xExp(:,i,t);
    end
end

yHat      = outKF.g0 + outKF.gx * outKF.xHat;
ny        = length(model.matSelect);
rExpAvg   = nan(ny, T);
for t = 1:T
    for i = 1:ny
        rExpAvg(i,t) = mean(rExp(1:model.matSelect(i), t));
    end
end

res.yHat      = yHat';
res.rExp      = rExpAvg';
res.termPremia = res.yHat - res.rExp;
res.matSelect = model.matSelect;
end

function res = local_campbellSchiller(Data, m)

[T, n]      = size(Data);
matSelect_CS = m : m : n*m;
shortRate   = Data(1:T-m, 1);
numYields   = n;

CSBetta     = NaN(2, numYields);
CSBetta_tstat = NaN(2, numYields);
CSBetta_se   = NaN(2, numYields);

```

```

Ydata          = NaN(T-m, numYields);
Xdata          = NaN(T-m, numYields);
R2             = NaN(1, numYields);

for i = 1 : numYields - 1
    k = matSelect_CS(i+1);
    Y = Data(1+m:T, i) - Data(1:T-m, i+1);
    x = (Data(1:T-m, i+1) - shortRate);
    X = [ones(T-m, 1), x * m / (k - m)];
    resOLS = local_nwest(Y, X, m+1);
    CSBetta(:, i+1) = resOLS.beta;
    CSBetta_se(:, i+1) = resOLS.beta ./ resOLS.tstat;
    CSBetta_tstat(:, i+1) = resOLS.tstat;
    Ydata(:, i+1) = Y;
    Xdata(:, i+1) = x * m / (k - m);
    R2(:, i+1) = resOLS.rsqr;
end

res.CSBetta = CSBetta;
res.maturities = matSelect_CS;
res.CSBetta_se = CSBetta_se;
res.CSBetta_tstat = CSBetta_tstat;
res.CSBetta_CI95 = [CSBetta(2,:) - 1.96 * CSBetta_se(2,:);
    ...
                    CSBetta(2,:) + 1.96 * CSBetta_se(2,:)];
res.CSBetta_CI99 = [CSBetta(2,:) - 2.575 * CSBetta_se(2,:);
    ...
                    CSBetta(2,:) + 2.575 * CSBetta_se(2,:)];
res.Ydata = Ydata;
res.Xdata = Xdata;
res.R2 = R2;
end

function res = local_nwest(y, X, lag)

```

```

% Newey-West OLS

[T, k] = size(X);
beta    = (X'*X) \ (X'*y);
e       = y - X*beta;

S = (e .* X)' * (e .* X);
for l = 1 : lag
    w    = 1 - l / (lag + 1);
    Xl    = X(l+1:T, :);
    el    = e(l+1:T);
    X0    = X(1:T-l, :);
    e0    = e(1:T-l);
    Gl    = (e0 .* X0)' * (el .* Xl);
    S     = S + w * (Gl + Gl');
end

XpX_inv = (X'*X) \ eye(k);
V        = T * XpX_inv * S * XpX_inv;
se       = sqrt(diag(V) / T);

res.beta  = beta;
res.tstat = beta ./ se;
res.se    = se;
res.rsqr  = 1 - var(e) / var(y);
end

% Extended Step 7 objective with three soft constraints:
% 1: Standard A2_star moment conditions (A-intercept fit)
% 2: Alpha anchor: alpha      mean_y1yr/ann
% 3: muQ shrinkage: |muQ| small
% 4: FULL UNCONDITIONAL LEVEL: embed Step 8 muP_1 recovery to
    penalise
%      deviations of the model-implied 10yr yield from its

```

```

    sample mean.
%
% The level constraint breaks the circular dependency between
    Steps 7 and 8
% by computing the implied muP_l forward from any candidate (
    alpha, muQ),
% then evaluating  $E[y_{10yr}] = g0_{10yr} + gx_{10yr} * E[x]$ .

function val = step7_objective(x, mature, phiQ, beta, sigma, ...
    A2_star, A1_star, B1l_ann,
    B2l_ann, B1l_chol, ...
    phiP_ll, IminusPhiP_ll, muP_m,
    ...
    nxM, nxL, ann, mean_y1yr, ...
    w_mean, w_muQ, w_level)

alpha_c = x(1);
muQ_c = [x(2:end); zeros(nxL,1)]; % nxM macro + nxL=0
    latent muQ

% Standard A2_star residuals
r_A2 = local_alphaMuQ_residuals(x, mature, phiQ, beta, sigma,
    ...
    A2_star, B1l_ann, B2l_ann,
    nxM, nxL, ann);

val = sum(r_A2.^2);

% Constraint 1: alpha anchor
val = val + w_mean^2 * (alpha_c - mean_y1yr/ann)^2;

% onstraint 2: muQ shrinkage
val = val + w_muQ^2 * sum(x(2:end).^2);

% Constraint 3: full unconditional 10yr level

```

```

% Recover muP_l implied by this (alpha, muQ) candidate (mirrors
  Step 8)
try
    [A1_ann_c, ~] = local_computeA_ann(alpha_c, muQ_c, phiQ,
        beta, sigma, ...
                                mature, nxM, nxL, ann);
    muP_l_c = B1l_chol \ (A1_star - A1_ann_c + ...
        B1l_chol * phiP_ll / B1l_chol *
        A1_ann_c);
    muP_c    = [muP_m; muP_l_c];

% Unconditional factor means  $E[x] = (I - \phi P)^{-1} * \mu P$ 
% Use block structure:  $E[x_m] = (I - \phi P_{mm})^{-1} * \mu P_m$       0
%
%                                $E[x_l] = (I - \phi P_{ll})^{-1} * \mu P_l$ 
Ex_l = IminusPhiP_ll \ muP_l_c;      % nxL x 1

% Implied 10yr yield:  $g_0(10yr) + g_x(10yr, :) * E[x]$ 
% gx row index for 10yr: matSelect = [4,8,20,28,40,60]; 10
%   yr=40Q is index 5
gx_10yr = gx(5,:);      % 1 x nx (annualised)
g0_10yr = gx_10yr * zeros(nxM+nxL,1); % placeholder;
%   actual g0 not yet computed

% Approximate  $E[y_{10yr}]$  using only the factor mean
%   contribution:
%  $g_x(10yr) = ann * (-B(:,40)') / 40$  from the phiQ/beta
%   recursion.
% We penalise  $g_x_{latent} * E[x_l]$  the latent drift
%   contribution to
% the 10yr yield. If this is large, the level will be off.
% Compute  $B(:,40)$  directly (40 quarters = 10yr):
n10 = 40;
B_k = -beta;
for kk = 2:n10

```

```
        B_k = -beta + phiQ' * B_k;
    end
    gx_10yr_lat = ann * (-B_k(nxM+1:end)' / n10);    % 1 x nxL

    latent_drift_10yr = gx_10yr_lat * Ex_1;    % scalar (
        annualised decimal)
    val = val + w_level^2 * latent_drift_10yr^2;
catch
    val = val + 1e20;    % penalise if recovery fails
end
end
```